

1. Modélisation

Modélisation et proposition de résolution de problèmes classiques

2. Métaheuristiques

métaheuristiques à base de voisinage (descente, recuit simulé, méthode Tabou, ...) et de population (algorithmes évolutionnaires, colonies de fourmis, ...)

3. Programmation linéaire

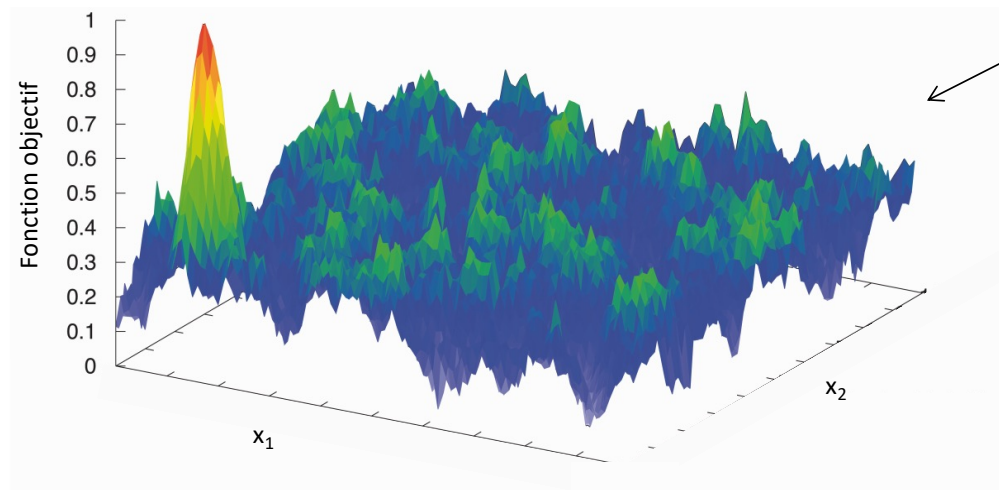
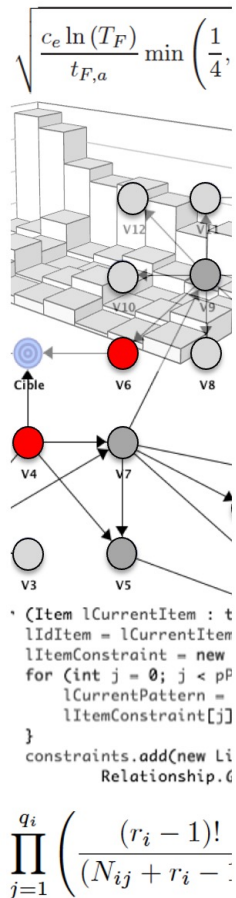
programme linéaire, méthode de résolution graphique, méthode algébrique, méthode du simplexe, dualité, programmation linéaire en nombres entiers

4. Optimisation multiobjectif

dominance de Pareto, métaheuristiques

PROBLÈME D'OPTIMISATION

Trouver une solution x_{opt} qui maximise (ou minimise) la fonction objectif dans l'espace de recherche.



Représentation hypothétique d'un **espace de recherche** sous forme d'un **paysage** où les **solutions** sont des points du plan positionnés les uns par rapport aux autres à l'aide d'une **structure de voisinage** et où la valeur de la **fonction objectif** est donnée par l'axe vertical.

« Complex Network Analysis of Fitness Landscapes. Sebastian Herrmann »

Comment se « balader » dans cet espace de recherche **discret** afin de trouver le **maximum global** de la fonction (ou plutôt **une bonne solution**) sans rester « coincé » dans les **maxima locaux** ?

PROGRAMMATION LINÉAIRE



Considérons une usine où il est possible d'assembler deux modèles de voitures.



On peut produire 100 voitures du premier type en 6 heures et 100 voitures du second type en 5 heures. Le nombre d'heures de travail est au maximum de 60 heures par semaine.

Les voitures produites sont enlevées une fois par semaine et doivent être stockées dans un dépôt de 15 000m². Une voiture du type 1 occupe 10m² et une voiture du type 2 occupe 20 m².

La marge sur le premier type de voiture est de 5000 € par véhicule et de 4500 € par véhicule pour le type 2. La demande pour le premier type de voiture est limitée à 800 unités par semaine ; la demande pour le deuxième type est tellement forte qu'on peut la considérer comme illimitée.

Combien de voitures de chaque type le constructeur doit-il produire par semaine pour maximiser son profit net ?

← on trouve généralement les variables dans la question

x_1 : nombre de véhicules de type 1

x_2 : nombre de véhicules de type 2

$$\frac{c_e \ln(T_F)}{t_{F,a}} \min\left(\frac{1}{4}, \dots\right)$$

```

    (Item lCurrentItem : t
    lIdItem = lCurrentItem
    lItemConstraint = new
    for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
    }
    constraints.add(new Li
    Relationship.6
  
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - \dots} \right)$$

PROGRAMMATION LINÉAIRE



Considérons une usine où il est possible d'assembler deux modèles de voitures.



On peut produire 100 voitures du premier type en 6 heures et 100 voitures du second type en 5 heures. Le nombre d'heures de travail est au maximum de 60 heures par semaine.

Les voitures produites sont enlevées une fois par semaine et doivent être stockées dans un **dépôt de 15 000m²**. Une voiture du type 1 occupe 10m² et une voiture du type 2 occupe 20 m².

$$x_1 + 2x_2 \leq 1500$$

La marge sur le premier type de voiture est de 5000 € par véhicule et de 4500 € par véhicule pour le type 2. La demande pour le premier type de voiture est limitée à 800 unités par semaine ; la demande pour le deuxième type est tellement forte qu'on peut la considérer comme illimitée.

Combien de voitures de chaque type le constructeur doit-il produire par semaine pour maximiser son profit net ?

x_1 : nombre de véhicules de type 1

x_2 : nombre de véhicules de type 2

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```

    (Item lCurrentItem : t
     lIdItem = lCurrentItem
     lItemConstraint = new
     for (int j = 0; j < pP
     lCurrentPattern =
     lItemConstraint[j])
     }
     constraints.add(new Li
     Relationship.
  
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

PROGRAMMATION LINÉAIRE



Considérons une usine où il est possible d'assembler deux modèles de voitures.



On peut produire 100 voitures du premier type en 6 heures et 100 voitures du second type en 5 heures. Le nombre d'heures de travail est au maximum de 60 heures par semaine.

$$6x_1 + 5x_2 \leq 6000$$

Les voitures produites sont enlevées une fois par semaine et doivent être stockées dans un dépôt de 15 000m². Une voiture du type 1 occupe 10m² et une voiture du type 2 occupe 20 m².

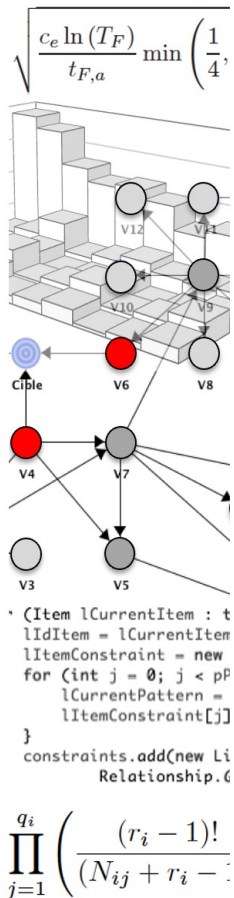
$$x_1 + 2x_2 \leq 1500$$

La marge sur le premier type de voiture est de 5000 € par véhicule et de 4500 € par véhicule pour le type 2. La demande pour le premier type de voiture est limitée à 800 unités par semaine ; la demande pour le deuxième type est tellement forte qu'on peut la considérer comme illimitée.

Combien de voitures de chaque type le constructeur doit-il produire par semaine pour maximiser son profit net ?

x_1 : nombre de véhicules de type 1

x_2 : nombre de véhicules de type 2



PROGRAMMATION LINÉAIRE



Considérons une usine où il est possible d'assembler deux modèles de voitures.



On peut produire 100 voitures du premier type en 6 heures et 100 voitures du second type en 5 heures. Le nombre d'heures de travail est au maximum de 60 heures par semaine.

$$6x_1 + 5x_2 \leq 6000$$

Les voitures produites sont enlevées une fois par semaine et doivent être stockées dans un dépôt de 15 000m². Une voiture du type 1 occupe 10m² et une voiture du type 2 occupe 20 m².

$$x_1 + 2x_2 \leq 1500$$

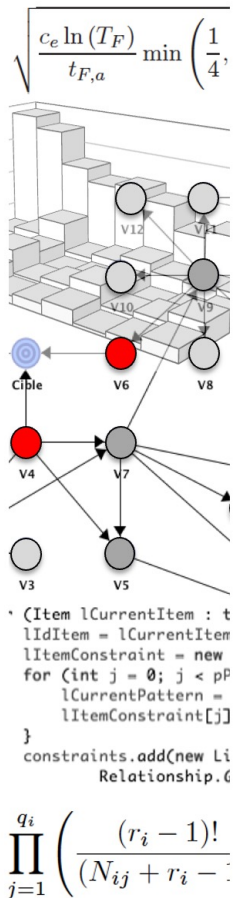
La marge sur le premier type de voiture est de 5000 € par véhicule et de 4500 € par véhicule pour le type 2. La demande pour le premier type de voiture est limitée à 800 unités par semaine ; la demande pour le deuxième type est tellement forte qu'on peut la considérer comme illimitée.

$$x_1 \leq 800$$

Combien de voitures de chaque type le constructeur doit-il produire par semaine pour maximiser son profit net ?

x_1 : nombre de véhicules de type 1

x_2 : nombre de véhicules de type 2



PROGRAMMATION LINÉAIRE



Considérons une usine où il est possible d'assembler deux modèles de voitures.



On peut produire 100 voitures du premier type en 6 heures et 100 voitures du second type en 5 heures. Le nombre d'heures de travail est au maximum de 60 heures par semaine.

$$6x_1 + 5x_2 \leq 6000$$

Les voitures produites sont enlevées une fois par semaine et doivent être stockées dans un dépôt de 15 000m². Une voiture du type 1 occupe 10m² et une voiture du type 2 occupe 20 m².

$$x_1 + 2x_2 \leq 1500$$

La marge sur le premier type de voiture est de 5000 € par véhicule et de 4500 € par véhicule pour le type 2. La demande pour le premier type de voiture est limitée à 800 unités par semaine ; la demande pour le deuxième type est tellement forte qu'on peut la considérer comme illimitée.

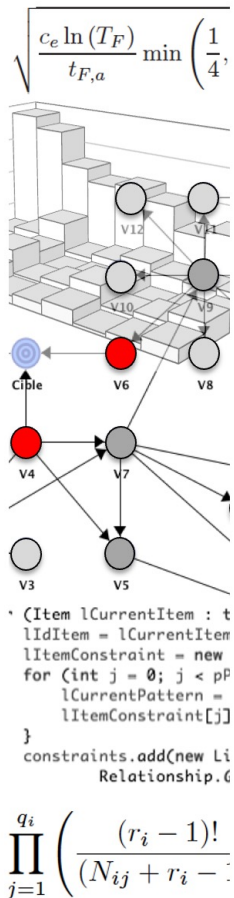
$$x_1 \leq 800$$

Combien de voitures de chaque type le constructeur doit-il produire par semaine pour maximiser son profit net ?

$$5000x_1 + 4500x_2 \text{ à maximiser}$$

x_1 : nombre de véhicules de type 1

x_2 : nombre de véhicules de type 2



PROGRAMMATION LINÉAIRE

Modélisation par un **programme linéaire (PL)** :

Variables

x_1 : nombre de voitures de type 1 produites ($x_1 \geq 0$)

x_2 : nombre de voitures de type 2 produites ($x_2 \geq 0$)

Contraintes

heures : $6x_1 + 5x_2 \leq 6000$

m^2 : $x_1 + 2x_2 \leq 1500$

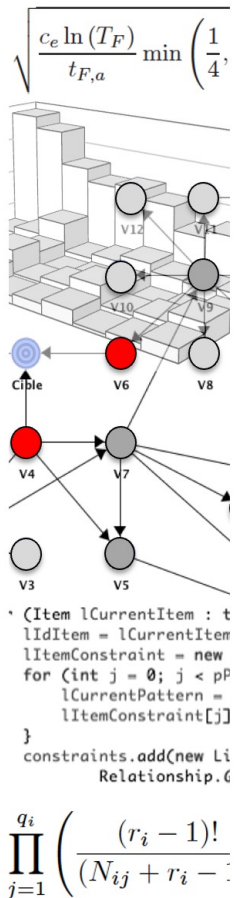
limite : $x_1 \leq 800$

Marge

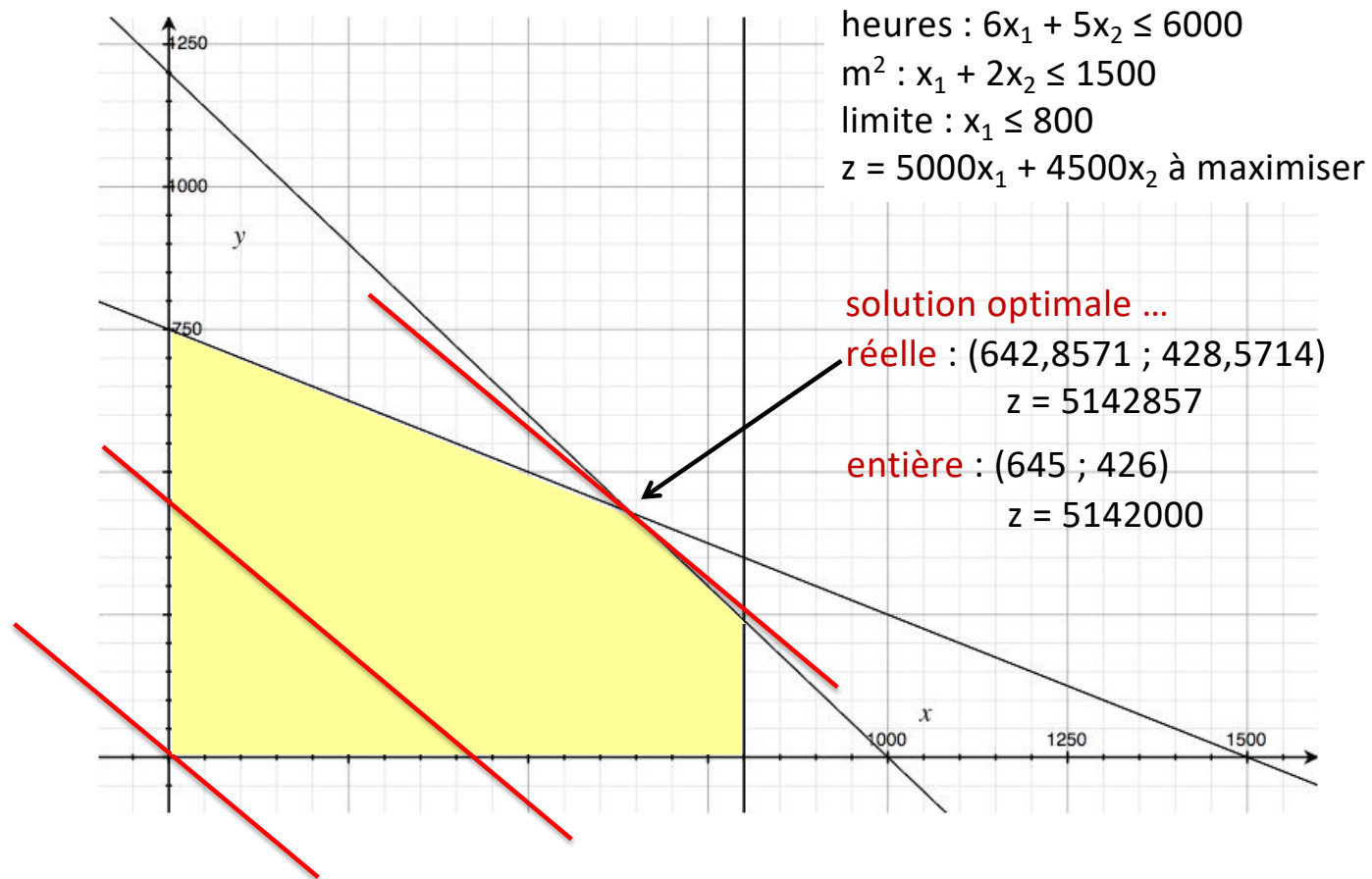
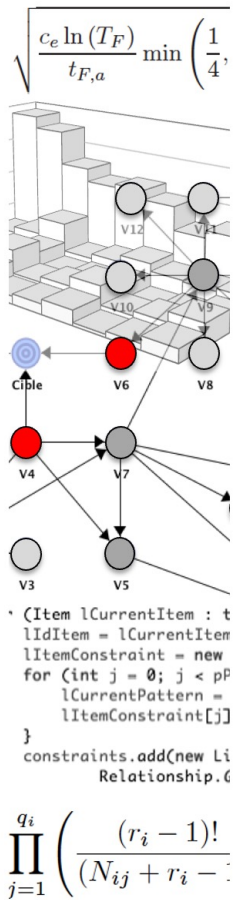
$z = 5000x_1 + 4500x_2$ à maximiser

Résolution graphique, algébrique, Simplexe, ...

Plus difficile à résoudre si les variables sont dans IN que dans IR !



PROGRAMMATION LINÉAIRE



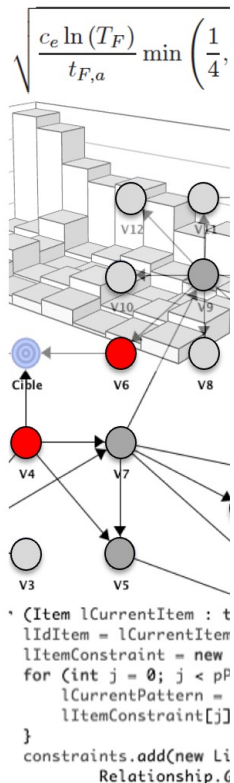
INVESTISSEMENT FINANCIER



Un business angel désire investir dans plusieurs entreprises innovantes. Il dispose de 100 k€. Il a identifié 5 start-up potentiellement très intéressantes. Pour chaque start-up, la somme à investir et l'espérance de gain à 5 ans ont été calculées et estimées, et sont données dans le tableau suivant :

Start-up	Investissement	Espérance de gain
S ₁	75 k€	210 k€
S ₂	43 k€	90 k€
S ₃	56 k€	300 k€
S ₄	12 k€	30 k€
S ₅	31 k€	70 k€

Comment le business angel doit-il répartir son investissement pour maximiser son espérance de gain à 5 ans (l'investissement dans une start-up doit se faire dans sa totalité) ?



$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

INVESTISSEMENT FINANCIER

Modélisation par un **programme linéaire en nombres entiers (PLNE)** :
binaires

Variables

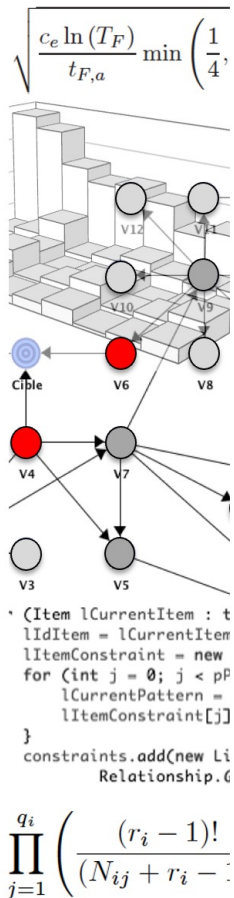
$x_i = 1$ si la start-up s_i est sélectionnée
 $= 0$ sinon

Contrainte

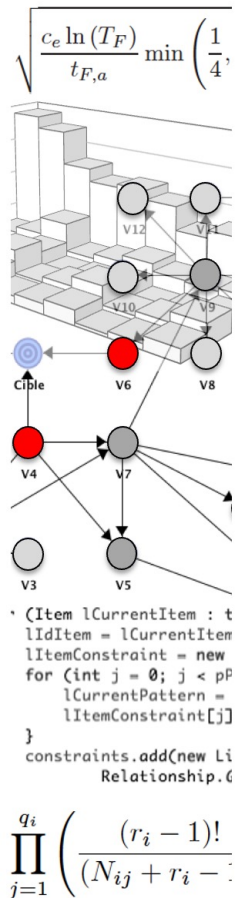
$75x_1 + 43x_2 + 56x_3 + 12x_4 + 31x_5 \leq 100 \text{ k€}$

Gain

$z = 210x_1 + 90x_2 + 300x_3 + 30x_4 + 70x_5$ à maximiser



INVESTISSEMENT FINANCIER



```

from ortools.sat.python import cp_model

model = cp_model.CpModel()

# variables
x = []
for i in range(5):
    x.append(model.NewBoolVar(f"x_{i}"))

# Contraintes
model.Add(75*x[0]+43*x[1]+56*x[2]+12*x[3]+31*x[4]<=100)

# Fonction objectif
c = [210,90,300,30,70]
model.Maximize(sum((c[i]*x[i]) for i in range(5)))

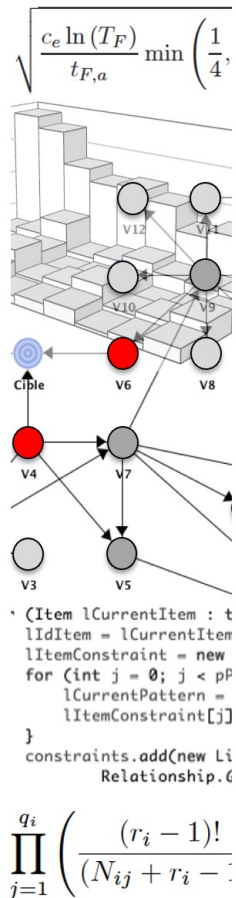
# Solution
solver = cp_model.CpSolver()
status = solver.Solve(model)
if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
    for i in range(5):
        print(f"x_{i+1} = " + str(solver.Value(x[i])))
        print("z = " + str(solver.ObjectiveValue()))
    else:
        print('No solution')
  
```

$$75x_1 + 43x_2 + 56x_3 + 12x_4 + 31x_5 \leq 100 \text{ k€}$$

$$z = 210x_1 + 90x_2 + 300x_3 + 30x_4 + 70x_5 \text{ à maximiser}$$

$$\begin{aligned} x_1 &= 0 \\ x_2 &= 0 \\ x_3 &= 1 \\ x_4 &= 1 \\ x_5 &= 1 \\ z &= 400.0 \end{aligned}$$

PROBLÈME DU SAC À DOS – KNAPSACK PROBLEM



Soient n objets ayant chacun un **poids** w_i et une **valeur d'utilité** u_i (positive)

Objectif : déterminer le sous-ensemble d'objets, à mettre dans le sac à dos, qui **maximise la somme des utilités des objets**

Contraintes : ne pas dépasser le **poids maximal** autorisé W dans le sac à dos

$$\max \sum_{i=1}^n u_i x_i \quad \sum_{i=1}^n w_i x_i \leq W \quad x_i \in \{0,1\}$$

Nombre de solutions potentielles :
nombre de sous-ensembles d'objets = 2^n



PROBLÈME DU SAC À DOS – KNAPSACK PROBLEM



	s_1	s_2	s_3	s_4	s_5
w_i	75	43	56	12	31
u_i	210	90	300	30	70

$W = 100 \text{ k€}$

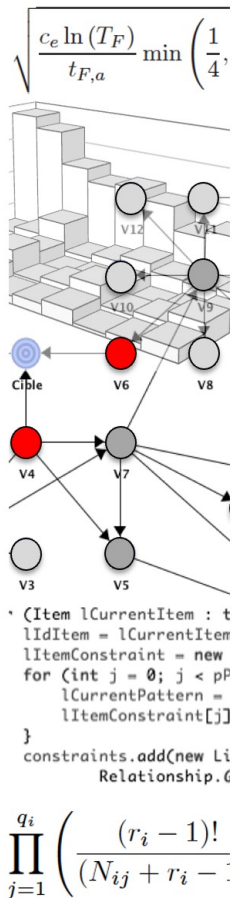
$$75x_1 + 43x_2 + 56x_3 + 12x_4 + 31x_5 \leq 100 \text{ k€}$$

$$z = 210x_1 + 90x_2 + 300x_3 + 30x_4 + 70x_5 \text{ à maximiser}$$

Sur un problème de taille raisonnable, il est possible d'imaginer une heuristique dite « constructive ».

Par exemple :

ajouter en priorité les start-up ayant le meilleur rapport utilité/poids
jusqu'à ce que le sac soit rempli



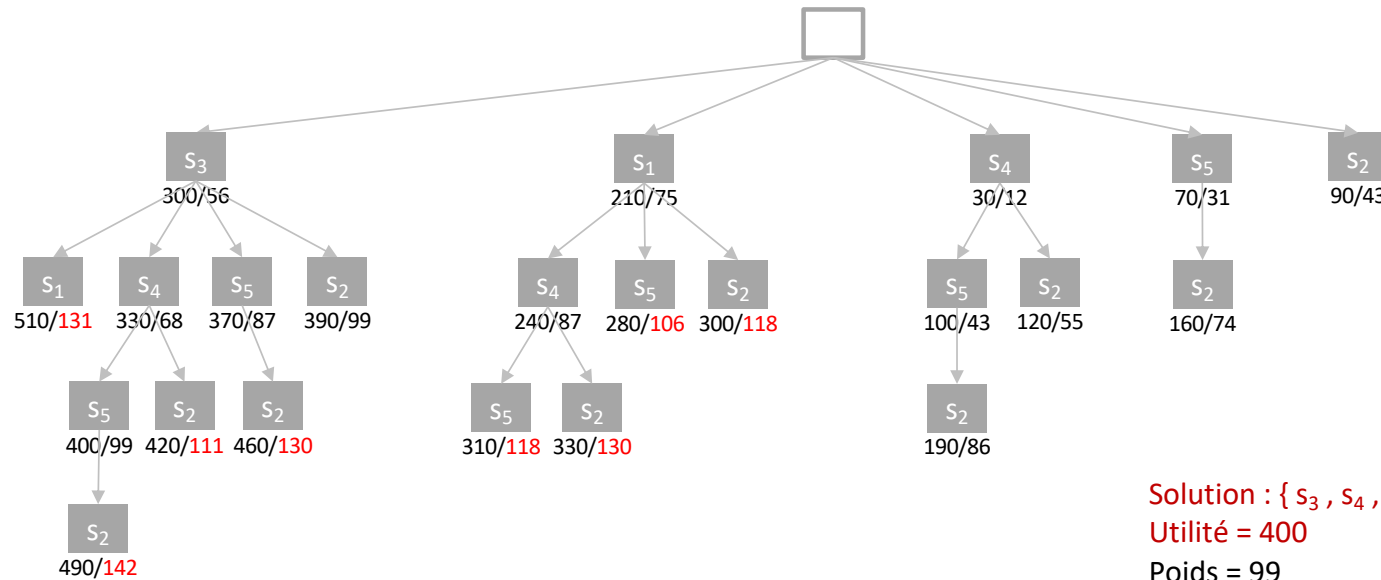
PROBLÈME DU SAC À DOS – KNAPSACK PROBLEM



	s_1	s_2	s_3	s_4	s_5
w_i	75	43	56	12	31
u_i	210	90	300	30	70
u_i / w_i	2,80	2,09	5,36	2,50	2,26

$W = 100 \text{ k€}$

$\rightarrow s_3 > s_1 > s_4 > s_5 > s_2$



Solution : $\{ s_3, s_4, s_5 \}$

Utilité = 400

Poids = 99

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

$(\text{Item } l\text{CurrentItem} : t$
 $l\text{IdItem} = l\text{CurrentItem}$
 $l\text{ItemConstraint} = \text{new}$
 $\text{for } (\text{int } j = 0; j < pP$
 $l\text{CurrentPattern} =$
 $l\text{ItemConstraint}[j]$
 $\}$
 $\text{constraints.add(new Li}$
 Relationship.

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

SUBSET SUM PROBLEM

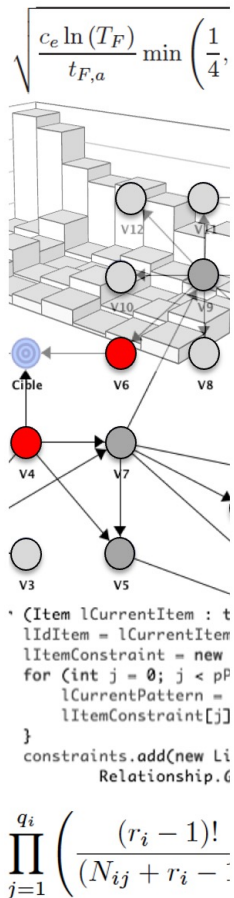
Soient n entiers et S une valeur entière

Objectif : trouver un sous-ensemble d'entiers dont la somme est égale à S

Exemple avec 30 entiers de $[0 ; 2^{60}-1]$:

483461097057306643	782691952487948605	719273246460721597	363044726835268177
782517498153186992	302676040615821699	565013932913844360	402049528275699057
722732610238437627	440554169739034657	181442816739714669	463897262338881917
1126819544586877267	1109683724713698202	952384372234545518	237867627831230139
224911549500469805	383889858300620166	913028127158345581	853952312197970642
488360996114439110	115421826238062736	783467347468775667	1143532530241779859
1003126349318295853	1021702344873866928	371324424707722660	519503375340169968
1031892357106197887	1068796830411924087		

Target sum : **8174987705823647296**



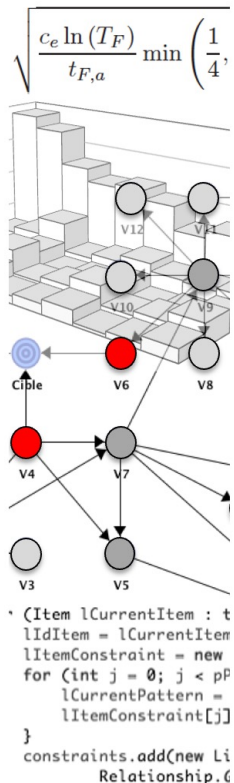
INVESTISSEMENT FINANCIER



Un business angel désire investir dans plusieurs entreprises innovantes. Il dispose de 100 k€. Il a identifié 5 start-up potentiellement très intéressantes. Pour chaque start-up, la somme à investir et l'espérance de gain à 5 ans ont été calculées et estimées, et sont données dans le tableau suivant :

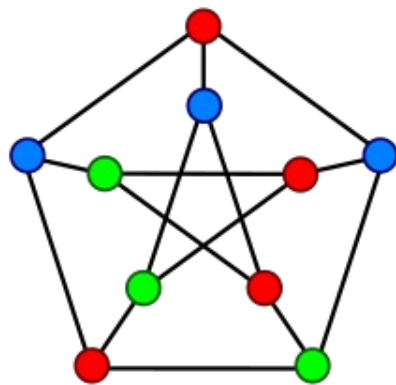
Start-up	Investissement	Espérance de gain
S ₁	75 k€	210 k€
S ₂	43 k€	90 k€
S ₃	56 k€	300 k€
S ₄	12 k€	30 k€
S ₅	31 k€	70 k€

Comment le business angel doit-il répartir son investissement pour maximiser son espérance de gain à 5 ans (l'investissement dans une start-up peut se faire partiellement) ?



$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

3-COLORATION

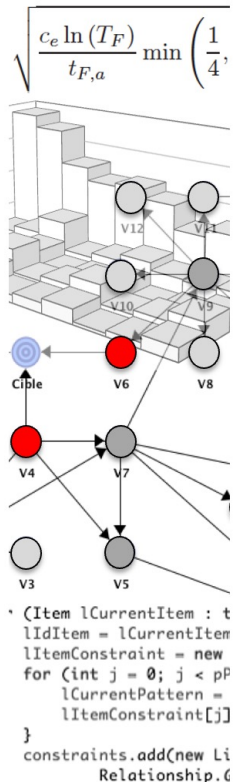


Soit $G=(V,E)$ un graphe.

L'objectif est de **modéliser par un programme linéaire en nombres entiers** le problème de 3-coloration du graphe (colorier les sommets du graphe de telle manière à ce que deux sommets adjacents n'aient pas la même couleur, et avec le moins de couleurs possibles).

Les 3 couleurs possibles sont représentées par les entiers : 1, 2 et 3.

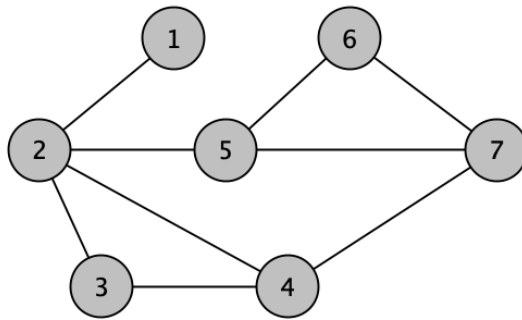
1. Quelles sont les variables de ce PLNE ?
2. Quelle est la contrainte linéaire qui garantit qu'un sommet v reçoit exactement une couleur ?
3. Quelles sont les contraintes linéaires qui garantissent qu'une arête (u,v) n'a pas ses deux extrémités colorées avec la même couleur ?
4. Proposer une fonction objectif (privilégier les plus petites couleurs).



$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

3-COLORATION

Implémentation en Python de la 3-coloration en utilisant le PLNE précédent



```
from ortools.sat.python import cp_model
model = cp_model.CpModel()
```

```
# Variables : v[i][c]
v = []
for i in range(graph.nbVertices):
    v_c = []
    for c in range(3):
        v_c.append(model.NewBoolVar(f"v_{i}{c+1}"))
    v.append(v_c)
```

```
# Contrainte n°1 : 1 sommet = 1 couleur
for i in range(graph.nbVertices):
    model.Add(v[i][0]+v[i][1]+v[i][2]==1)
```

```
# Contrainte n°2 : 1 arête = 3 contraintes
for i in range(graph.nbEdges):
    j = graph.listEdges[i].idVertexInitial
    k = graph.listEdges[i].idVertexFinal
    model.Add(v[j][0] + v[k][0] <= 1)
    model.Add(v[j][1] + v[k][1] <= 1)
    model.Add(v[j][2] + v[k][2] <= 1)
```

```
# Exemple de fonction objectif
model.Minimize(sum((v[i][0] + 10*v[i][1] + 100*v[i][2]) for i in range(graph.nbVertices)))
```

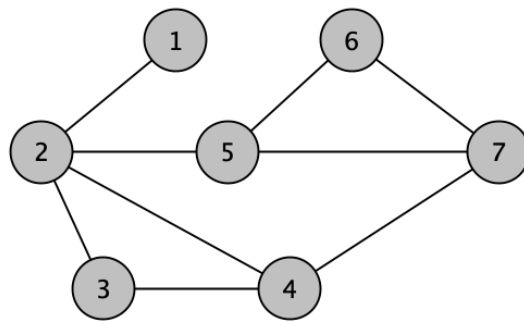
$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```
{
    (Item lCurrentItem : t
    lIdItem = lCurrentItem
    lItemConstraint = new
    for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
    }
    constraints.add(new Li
    Relationship.C
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)} \right)$$

3-COLORATION

Implémentation en Python de la 3-coloration en utilisant le PLNE précédent



Solution :

$v_1 = 1$
$v_2 = 3$
$v_3 = 2$
$v_4 = 1$
$v_5 = 1$
$v_6 = 2$
$v_7 = 3$

Contraintes :

$v_{11} + v_{12} + v_{13} == 1$	$v_{11} + v_{21} \leq 1$	$v_{41} + v_{71} \leq 1$
$v_{21} + v_{22} + v_{23} == 1$	$v_{12} + v_{22} \leq 1$	$v_{42} + v_{72} \leq 1$
$v_{31} + v_{32} + v_{33} == 1$	$v_{13} + v_{23} \leq 1$	$v_{43} + v_{73} \leq 1$
$v_{41} + v_{42} + v_{43} == 1$	$v_{21} + v_{31} \leq 1$	$v_{51} + v_{61} \leq 1$
$v_{51} + v_{52} + v_{53} == 1$	$v_{22} + v_{32} \leq 1$	$v_{52} + v_{62} \leq 1$
$v_{61} + v_{62} + v_{63} == 1$	$v_{23} + v_{33} \leq 1$	$v_{53} + v_{63} \leq 1$
$v_{71} + v_{72} + v_{73} == 1$	$v_{21} + v_{41} \leq 1$	$v_{51} + v_{71} \leq 1$
	$v_{22} + v_{42} \leq 1$	$v_{52} + v_{72} \leq 1$
	$v_{23} + v_{43} \leq 1$	$v_{53} + v_{73} \leq 1$
	$v_{21} + v_{51} \leq 1$	$v_{61} + v_{71} \leq 1$
	$v_{22} + v_{52} \leq 1$	$v_{62} + v_{72} \leq 1$
	$v_{23} + v_{53} \leq 1$	$v_{63} + v_{73} \leq 1$
	$v_{31} + v_{41} \leq 1$	
	$v_{32} + v_{42} \leq 1$	
	$v_{33} + v_{43} \leq 1$	

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```

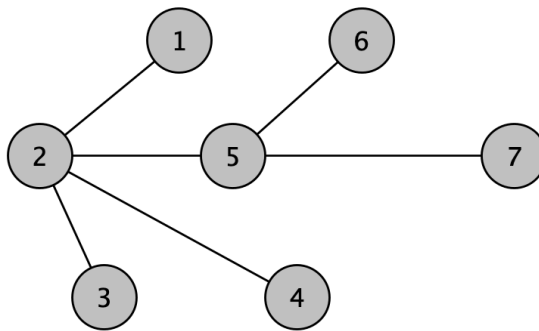
(Item lCurrentItem : t
 lIdItem = lCurrentItem
 lItemConstraint = new
 for (int j = 0; j < pP
 lCurrentPattern =
 lItemConstraint[j]
 }
 constraints.add(new Li
 Relationship.6

```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

3-COLORATION

Implémentation en Python de la 3-coloration en utilisant le PLNE précédent



Solution :

$v_1 = 1$
$v_2 = 2$
$v_3 = 1$
$v_4 = 1$
$v_5 = 1$
$v_6 = 2$
$v_7 = 2$

Contraintes :

$v_{11} + v_{12} + v_{13} == 1$	$v_{11} + v_{21} <= 1$	$v_{21} + v_{51} <= 1$
$v_{21} + v_{22} + v_{23} == 1$	$v_{12} + v_{22} <= 1$	$v_{22} + v_{52} <= 1$
$v_{31} + v_{32} + v_{33} == 1$	$v_{13} + v_{23} <= 1$	$v_{23} + v_{53} <= 1$
$v_{41} + v_{42} + v_{43} == 1$	$v_{21} + v_{31} <= 1$	$v_{51} + v_{61} <= 1$
$v_{51} + v_{52} + v_{53} == 1$	$v_{22} + v_{32} <= 1$	$v_{52} + v_{62} <= 1$
$v_{61} + v_{62} + v_{63} == 1$	$v_{23} + v_{33} <= 1$	$v_{53} + v_{63} <= 1$
$v_{71} + v_{72} + v_{73} == 1$	$v_{21} + v_{41} <= 1$	$v_{51} + v_{71} <= 1$
	$v_{22} + v_{42} <= 1$	$v_{52} + v_{72} <= 1$
	$v_{23} + v_{43} <= 1$	$v_{53} + v_{73} <= 1$

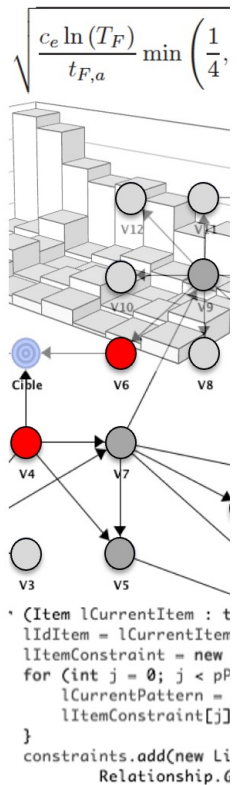
$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```

    (Item lCurrentItem : t
      lIdItem = lCurrentItem
      lItemConstraint = new
        for (int j = 0; j < pP
          lCurrentPattern =
            lItemConstraint[j]
        }
      constraints.add(new Li
        Relationship.6
    
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - \dots} \right)$$

PERÇAGE DE PLAQUE



$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

Une tête de perçage  doit percer n trous  dans une plaque rectangulaire.

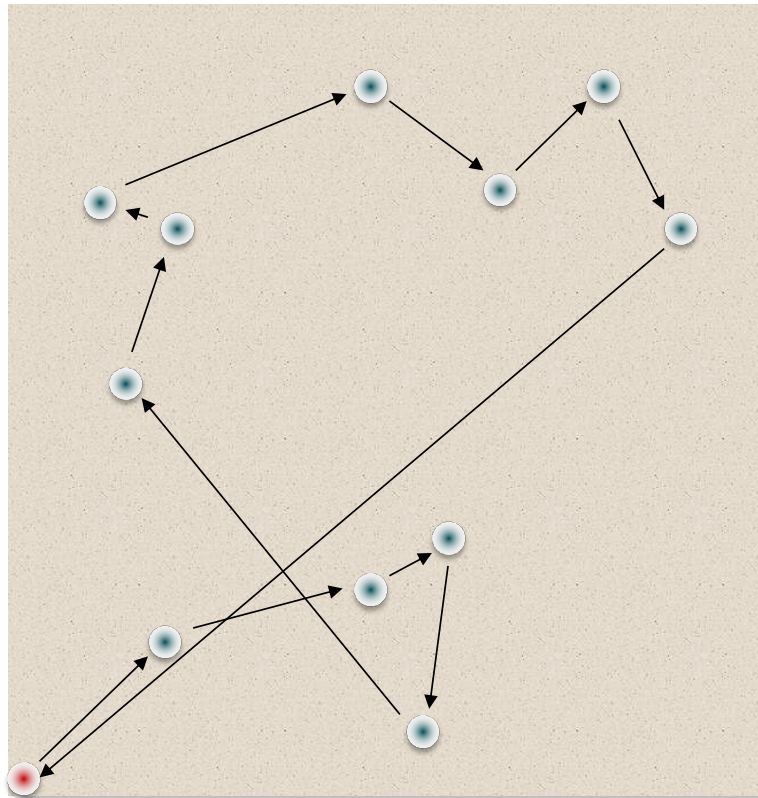
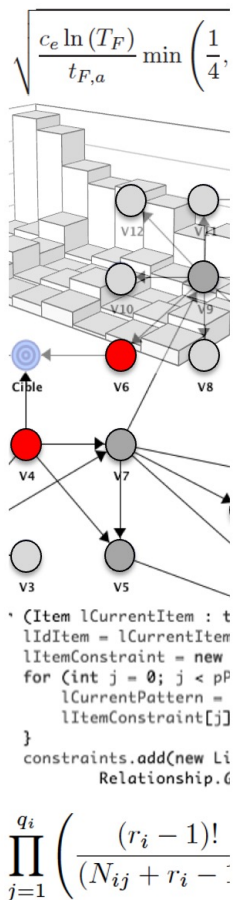
La tête de perçage est initialement en position (0,0).
On connaît la position (x_i, y_i) de chaque emplacement
de trou.

Percer un trou prend m secondes, tandis que le déplacement de la tête de perçage d'un emplacement à l'autre est proportionnel à la distance.

A la fin du perçage, la tête de perçage doit revenir à son point de départ.

Comment minimiser le temps total d'exécution ?

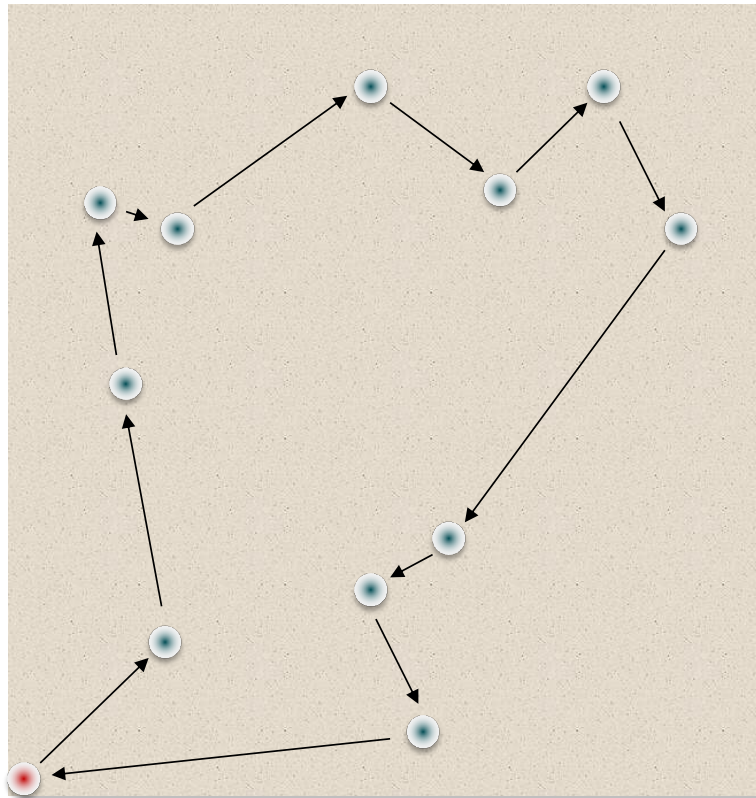
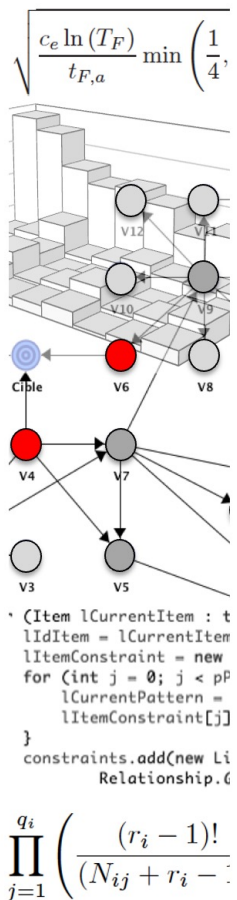
PERÇAGE DE PLAQUE



Méthode **gloutonne** (méthode qui ne remet pas en cause les choix fait à chaque étape de l'algorithme) intuitive :

« aller au plus proche »

PERÇAGE DE PLAQUE



Méthode gloutonne (méthode qui ne remet pas en cause les choix fait à chaque étape de l'algorithme) intuitive :

« aller au plus proche »

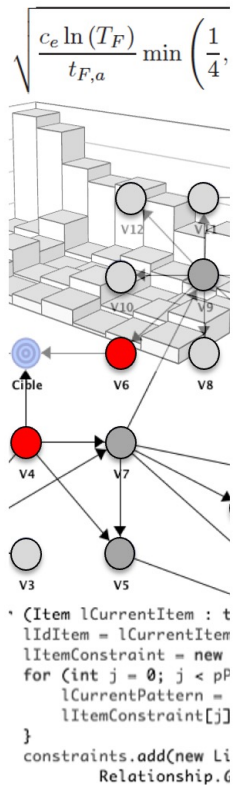
⇒ un choix **localement optimal**, ne garantit pas un résultat **globalement optimal**

Problème du voyageur de commerce Traveling salesman problem

PROBLÈME DU VOYAGEUR DE COMMERCE — TRAVELING SALESMAN PROBLEM

Soient n villes.

Comment trouver le trajet de longueur minimale passant par toutes les villes et revenant à la ville de départ ?



$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$



TSP Home

The Traveling Salesman Problem

The Traveling Salesman Problem is one of the most intensively studied problems in computational mathematics. These pages are devoted to the history, applications, and current research of this challenge of finding the shortest route visiting each member of a collection of locations and returning to your starting point.

How to solve the TSP! (YouTube), TSP cutting-plane method (YouTube or as mov)

- > Home
- The Problem
- History
- Applications
- Solving a TSP
- World Records
- Gallery
- TSP Games
- Concorde
- Test Data
- News
- TSP Book
- Search Site

Optimal crawl to 49,687 pubs in the UK. Visit 49,603 historic sites in the US.

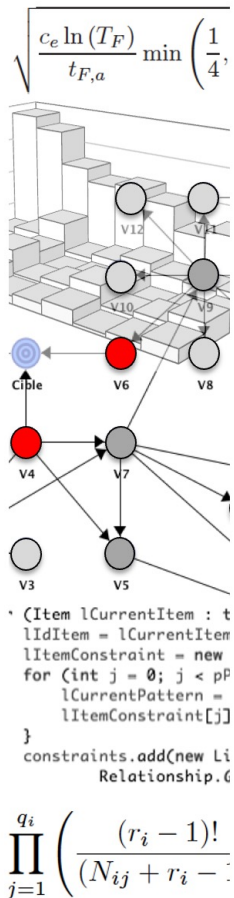
Connecting the Dots in the TSP (YouTube) 2018 Joint Math Meetings

<http://www.math.uwaterloo.ca/tsp/>

PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Soient n villes.

Comment trouver le trajet de longueur minimale passant par toutes les villes et revenant à la ville de départ ?



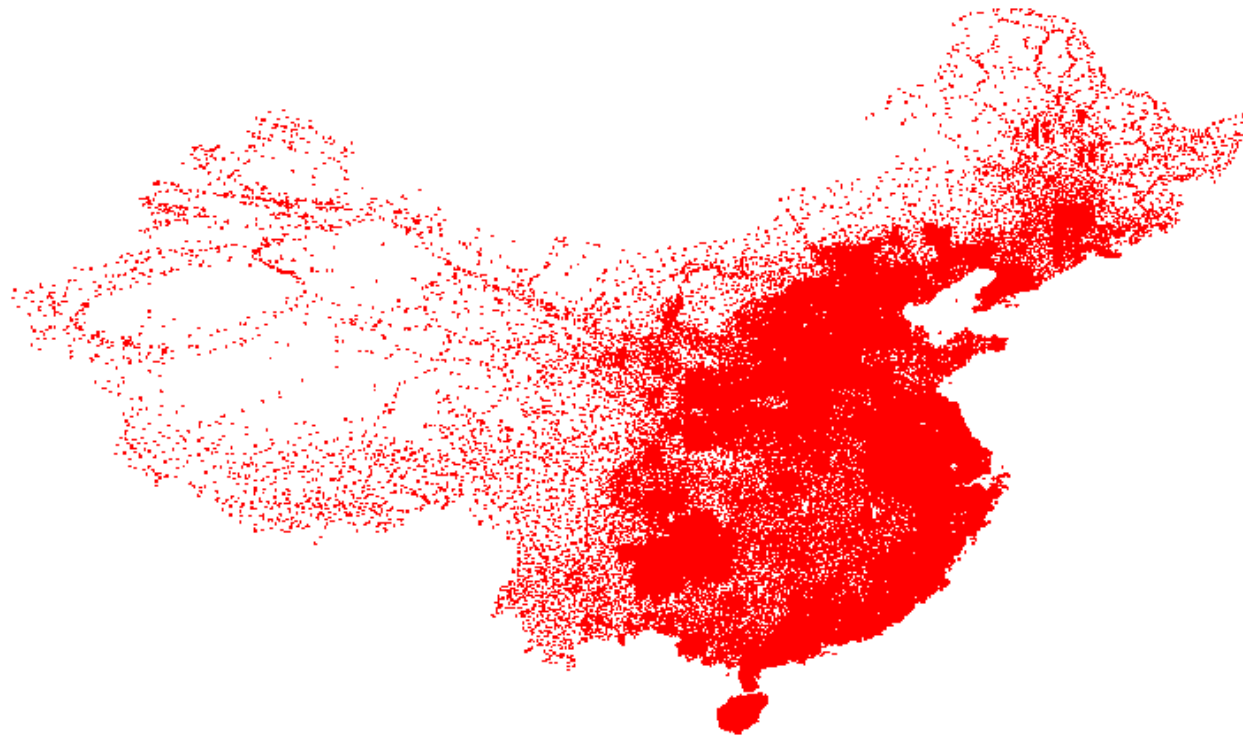
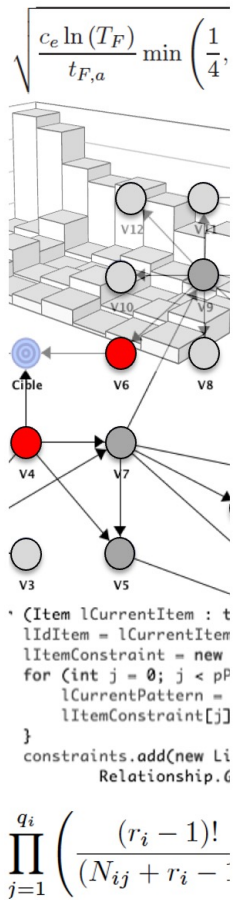
S = ensemble de tous les trajets possibles

$f(x)$ = longueur du trajet x

$$|S| = \frac{(n-1)!}{2} \quad \left| \begin{array}{l} \text{si } n=30 \\ \Rightarrow > \text{à } 4 \times 10^{30} \text{ trajets} \end{array} \right.$$

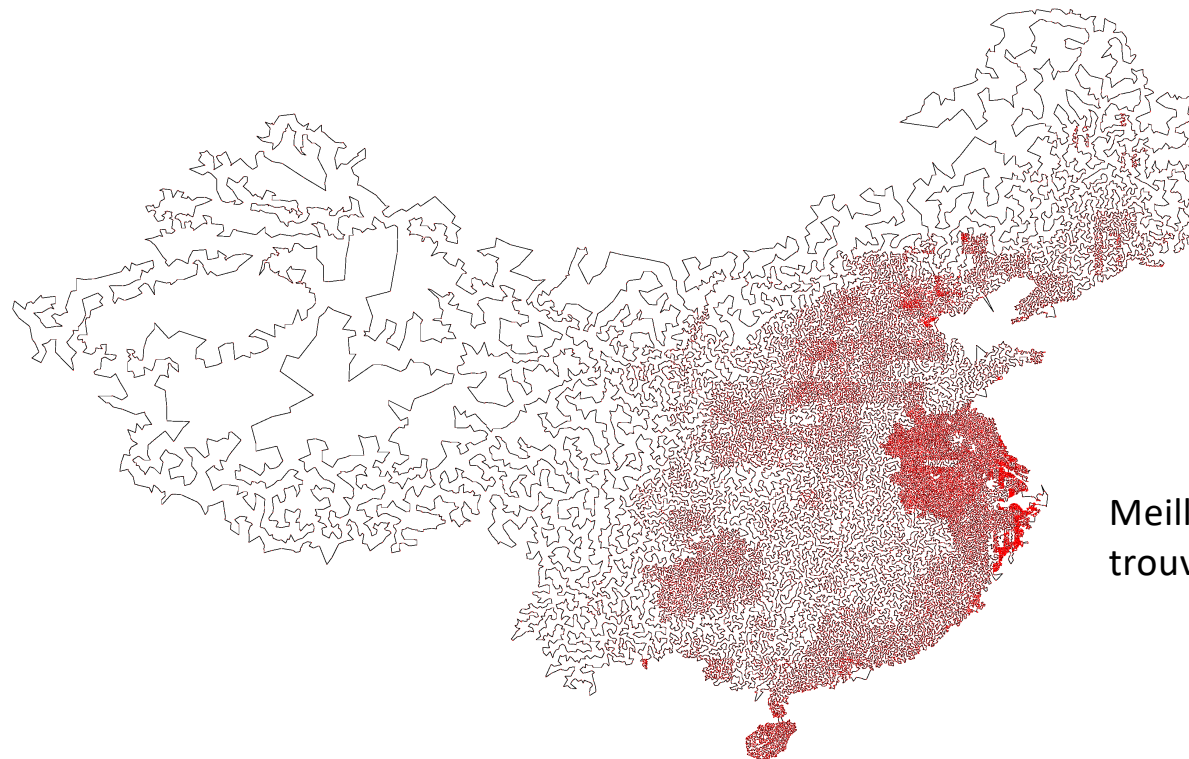
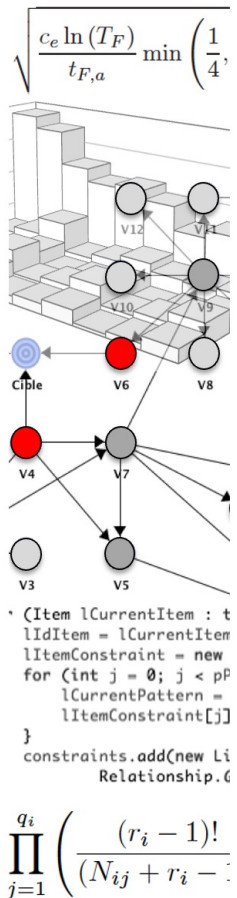
PROBLÈME DU VOYAGEUR DE COMMERCE — TRAVELING SALESMAN PROBLEM

Exemple avec 71009 villes chinoises



PROBLÈME DU VOYAGEUR DE COMMERCE — TRAVELING SALESMAN PROBLEM

Exemple avec 71009 villes chinoises



Meilleure solution
trouvée en juillet 2007

<https://www.math.uwaterloo.ca/tsp/world/chtour.html>

PROBLÈME DU VOYAGEUR DE COMMERCE — TRAVELING SALESMAN PROBLEM

Modélisation par un **PLNE** :

$x_{ij} = 1$ si l'arc (i,j) est présent dans le trajet et 0 sinon

Objectif :

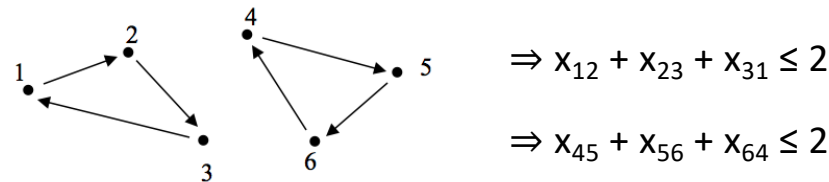
Minimiser la distance totale : $\min \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$ où c_{ij} = distance $i \rightarrow j$

Contraintes :

On arrive en j par un seul chemin : $\sum_{i=1}^n x_{ij} = 1, \forall j$

On quitte i par un seul chemin : $\sum_{j=1}^n x_{ij} = 1, \forall i$

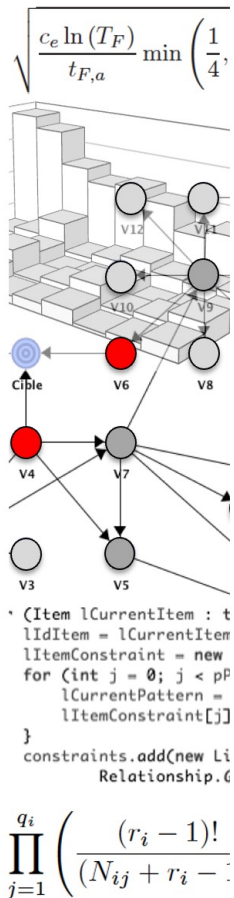
Il faut éviter les sous-trajets :



donc : $\sum_{(i,j) \in T} x_{ij} \leq |T|-1$, pour toute sous-tournée T

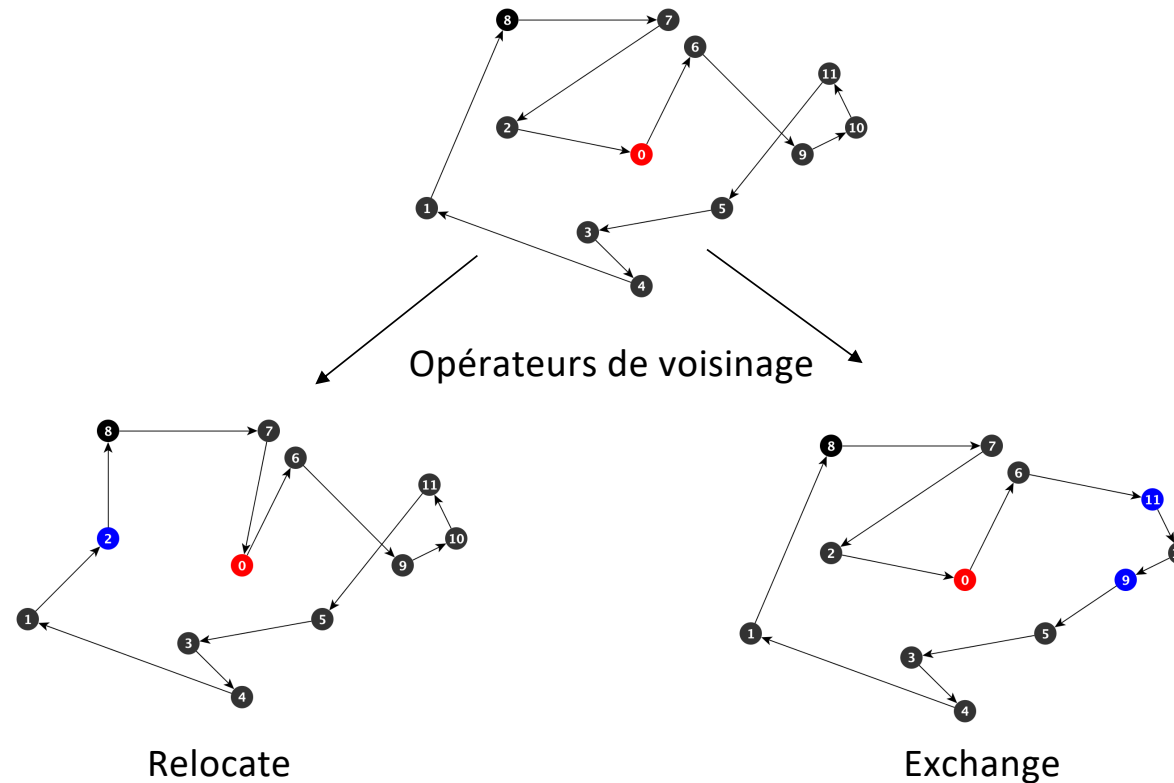
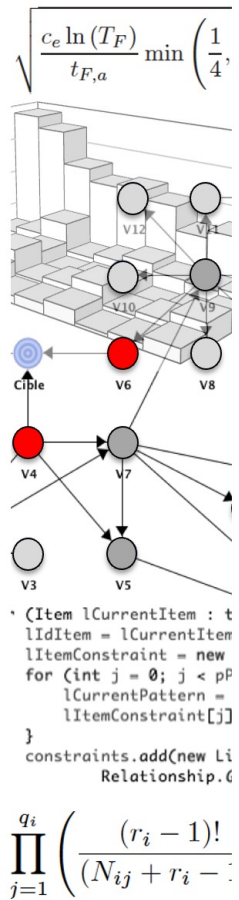
$\Rightarrow 2^n - 1$ contraintes supplémentaires

$\Rightarrow$ beaucoup trop dans la plupart des cas $\Rightarrow$ MILP



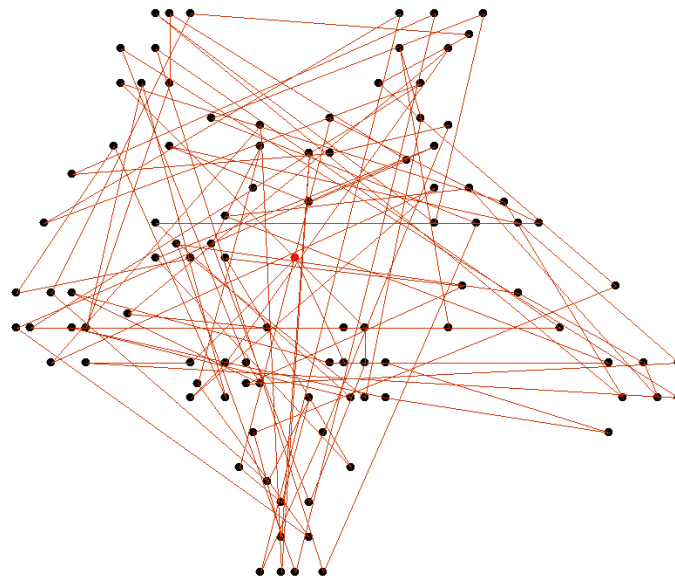
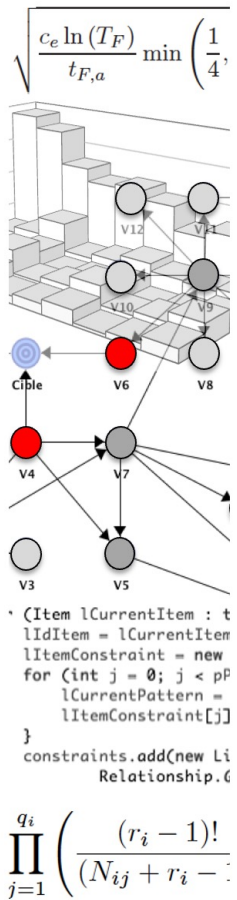
PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Résolution par **Métaheuristiques** – exemple de méthodes à base de voisinage :

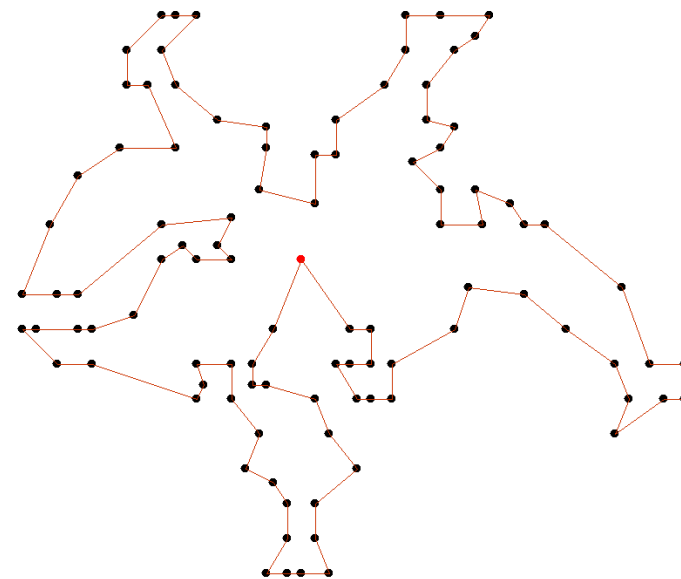


PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Résolution par **Métaheuristiques** – exemple de méthodes à base de voisinage :



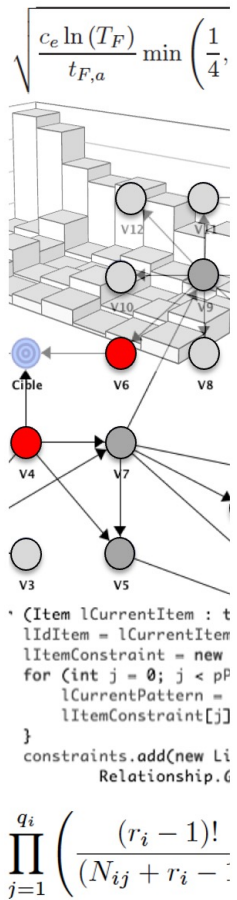
cvrp_C201 Fitness : 3993,37 - Nb Vehicles : 1



cvrp_C201 Fitness : 579,56 - Nb Vehicles : 1

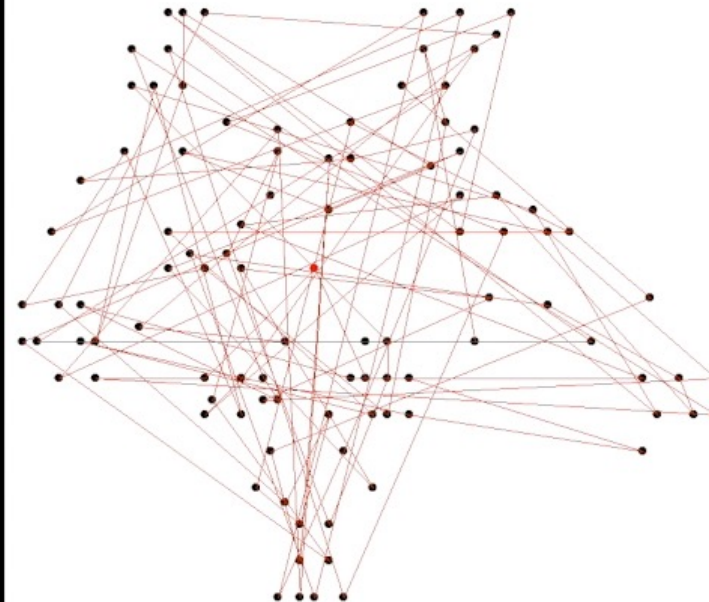
PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Résolution par **Métaheuristiques** – exemple de méthodes à base de voisinage :



TSP

Voisinage :
Relocate
Exchange
2-Opt

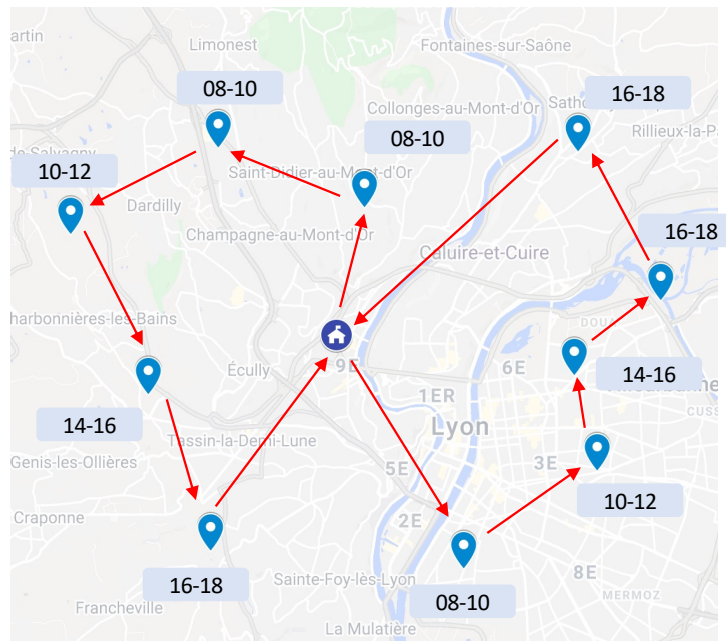
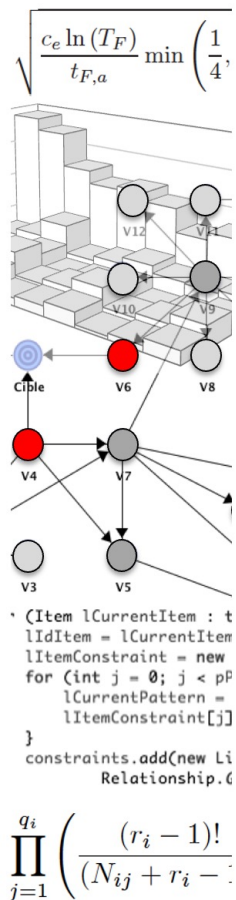


cvrp_C201 Fitness : 3993,37 - Nb Vehicles : 1

Résultat d'une
exécution à partir
d'une solution
initiale générée
aléatoirement

PROBLÈME DU VOYAGEUR DE COMMERCE — TRAVELING SALESMAN PROBLEM

Etant donnés n clients à livrer et k véhicules de livraison, déterminer la meilleure façon de livrer ces n clients à partir d'un dépôt avec ces k véhicules en minimisant la distance totale des trajets de tous les véhicules.



Données et contraintes :

- Coordonnées GPS des clients et du dépôt
- Plage horaire de livraison par client
- Nombre de colis par client
- Temps de livraison estimé par client
- Capacité maximum de chaque véhicule
- Contraintes horaires des chauffeurs

Objectif :

Organiser la tournée de chaque véhicule en respectant les plages horaires des clients, les capacités des véhicules et toutes les autres contraintes, tout en minimisant la distance totale.

PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Capacitated Vehicle Routing Problem

Clients + Dépôts : $C = \{0, 1, \dots, n, n+1\}$, Clients : $C^* = C - \{0, n+1\}$

Véhicules : $V = \{1, \dots, m\}$

Graphe : $G = (C, E)$ et E^* = arcs non incidents à 0 et $n+1$

Q : capacité maximum

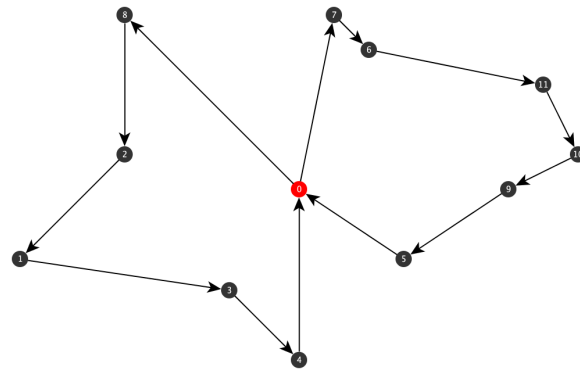
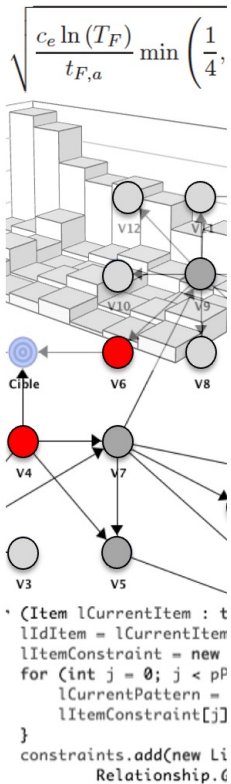
q_i : quantité de produits à livrer à $i \in C$ ($q_0 = q_{n+1} = 0$)

t_{ij} : temps de trajet associé à l'arc $(i, j) \in E$

c_{ij} : coût du trajet associé à l'arc $(i, j) \in E$

$x_{ij}^k = 1$ si l'arc $(i, j) \in E$ est présent dans la tournée du véhicule k , 0 sinon

u_i^k : quantité cumulée livrée par le véhicule k lorsqu'il quitte le lieu $i \in C$



$$\text{Minimiser } z = \sum_{k \in V} \sum_{i \in C} \sum_{j \in C} c_{ij} x_{ij}^k$$

$$\sum_{j \in C^*} x_{0j}^k = 1 \quad \forall k \in V$$

$$\sum_{i \in C^*} x_{i, n+1}^k = 1 \quad \forall k \in V$$

$$\sum_{i \in C, i \neq p} x_{ip}^k = \sum_{j \in C, j \neq p} x_{pj}^k \quad \forall p \in C^*, \forall k \in V$$

$$\sum_{k \in V} \sum_{i \in C, i \neq j} x_{ij}^k = 1 \quad \forall j \in C^*$$

$$\sum_{i \in C} \sum_{j \in C, i \neq j} q_i x_{ij}^k \leq Q \quad \forall k \in V$$

$$u_i^k + q_j - u_j^k \leq Q(1 - x_{ij}^k) \quad \forall (i, j) \in E, \forall k \in V$$

$$q_i \leq u_i^k \leq Q \quad \forall i \in C, \forall k \in V$$

$$x_{ij}^k \in \{0, 1\} \quad \forall (i, j) \in E, \forall k \in V$$

Ajout de variables non binaires pour éliminer les contraintes de sous-tournées

Mixed-integer linear programming (MILP)

PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Vehicle Routing Problem with Time Windows

Clients + Dépôts : $C = \{0, 1, \dots, n, n+1\}$, Clients : $C^* = C - \{0, n+1\}$

Véhicules : $V = \{1, \dots, m\}$

Graphe : $G = (C, E)$ et $E^* = \text{arcs non incidents à } 0 \text{ et } n+1$

Q : capacité maximum

q_i : quantité de produits à livrer à $i \in C$ ($q_0 = q_{n+1} = 0$)

d_i : durée de livraison en $i \in C$ ($d_0 = d_{n+1} = 0$)

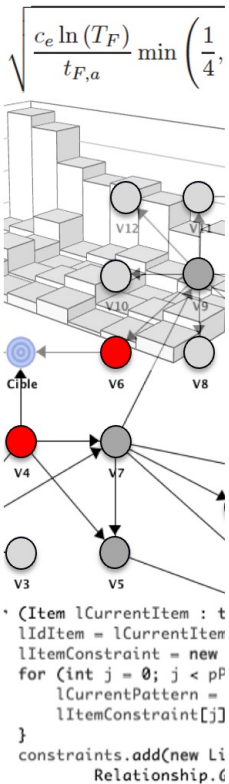
$[a_i; b_i]$: plage horaire dans laquelle la livraison doit débuter ($[a_0; b_0] = [a_{n+1}; b_{n+1}]$)

t_{ij} : temps de trajet associé à l'arc $(i, j) \in E$

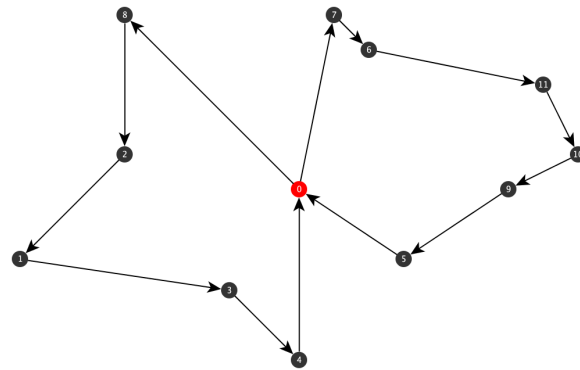
c_{ij} : coût du trajet associé à l'arc $(i, j) \in E$

$x_{ij}^k = 1$ si l'arc $(i, j) \in E$ est présent dans la tournée du véhicule k , 0 sinon

h_i^k : heure du début de la livraison du client $i \in C$ par le véhicule k



$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)} \right)$$



$$\text{Minimiser } z = \sum_{k \in V} \sum_{i \in C} \sum_{j \in C} c_{ij} x_{ij}^k$$

$$\sum_{j \in C^*} x_{0j}^k = 1 \quad \forall k \in V$$

$$\sum_{i \in C^*} x_{i, n+1}^k = 1 \quad \forall k \in V$$

$$\sum_{i \in C, i \neq p} x_{ip}^k = \sum_{j \in C, j \neq p} x_{pj}^k \quad \forall p \in C^*, \forall k \in V$$

$$\sum_{k \in V} \sum_{i \in C, i \neq j} x_{ij}^k = 1 \quad \forall j \in C^*$$

$$\sum_{i \in C} \sum_{j \in C, i \neq j} q_i x_{ij}^k \leq Q \quad \forall k \in V$$

$$h_i^k + d_i + t_{ij} - h_j^k \leq (1 - x_{ij}^k)M \quad \forall (i, j) \in E, \forall k \in V$$

$$a_i \leq h_i^k \leq b_i \quad \forall i \in C, \forall k \in V$$

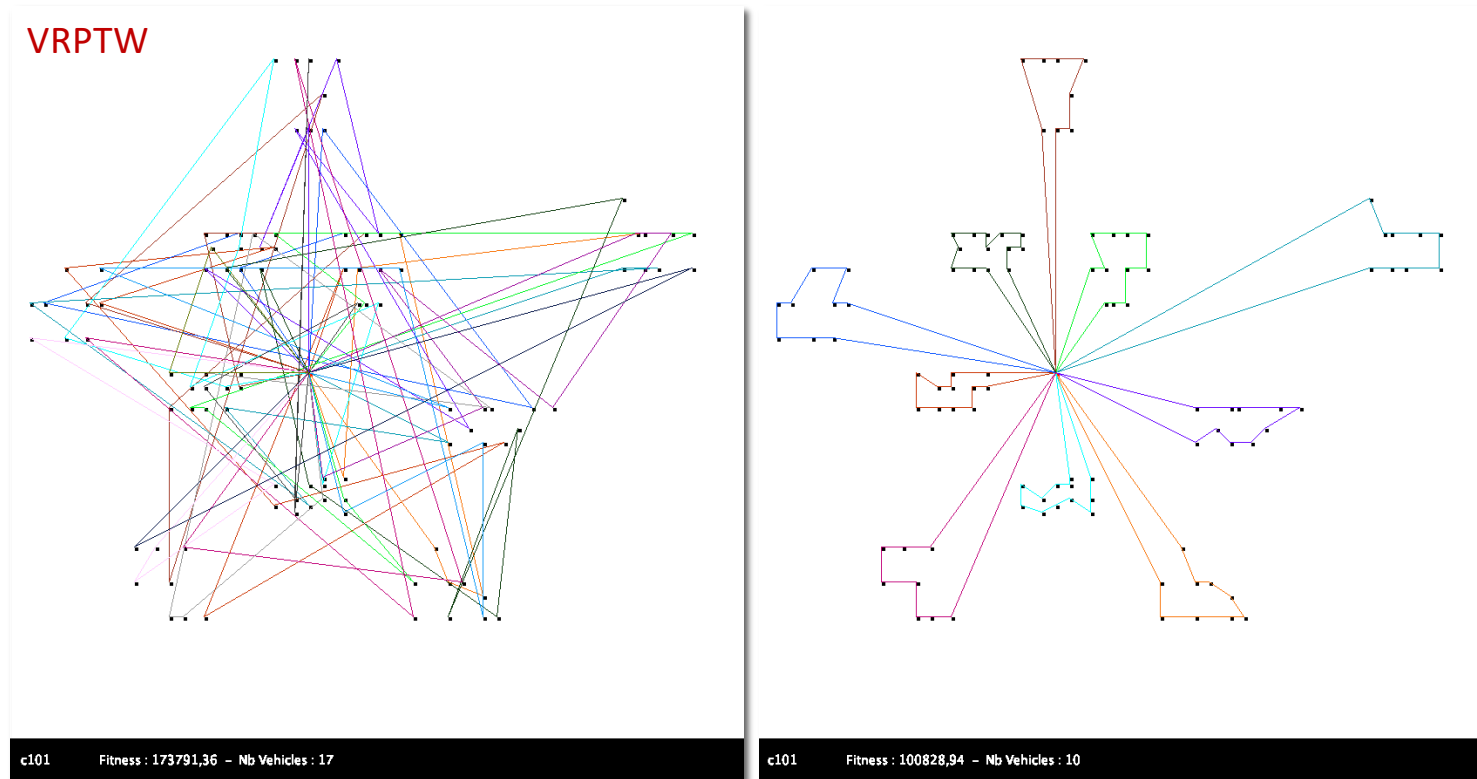
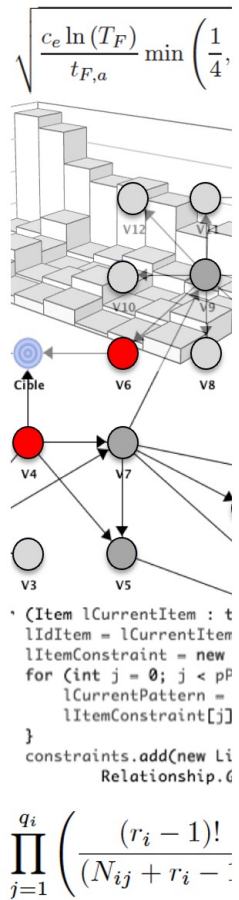
$$x_{ij}^k \in \{0, 1\} \quad \forall (i, j) \in E, \forall k \in V$$

Les contraintes horaires évitent les contraintes de sous-tournées

Mixed-integer linear programming (MILP)

PROBLÈME DU VOYAGEUR DE COMMERCE — TRAVELING SALESMAN PROBLEM

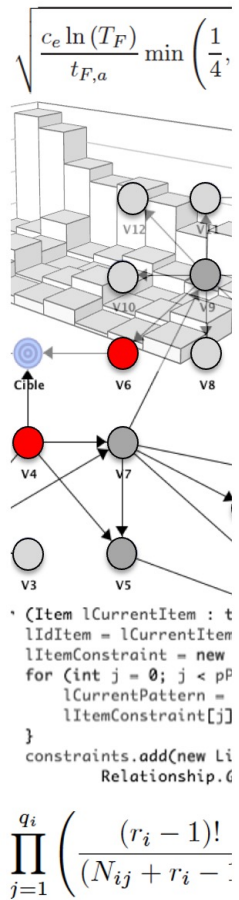
Résolution par **Métaheuristiques** (algorithmes génétiques, recuit simulé, ...):



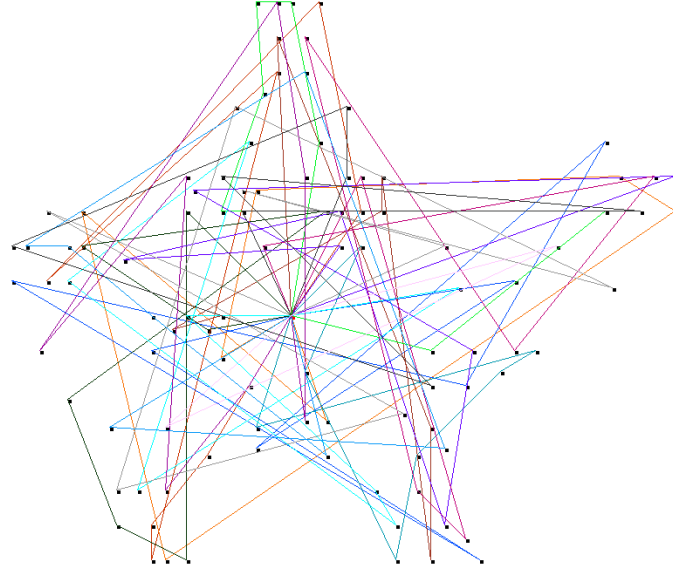
<https://www.sintef.no/projectweb/top/vrptw/solomon-benchmark/documentation/>

PROBLÈME DU VOYAGEUR DE COMMERCE — TRAVELING SALESMAN PROBLEM

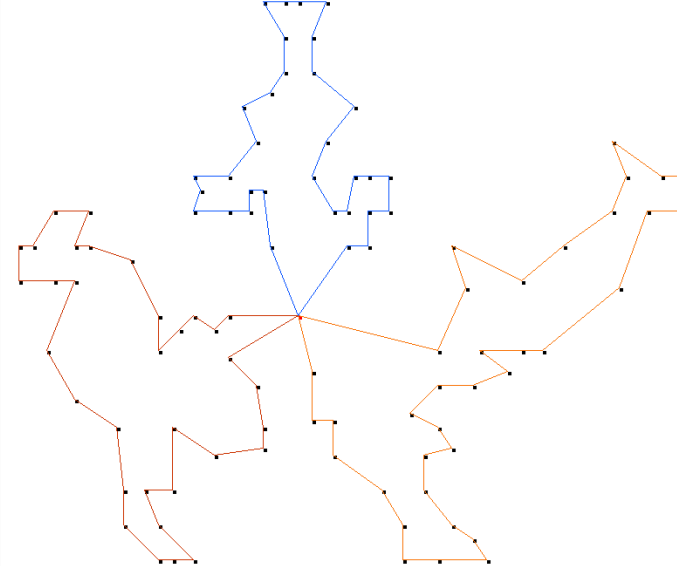
Résolution par **Métaheuristiques** (algorithmes génétiques, recuit simulé, ...) :



VRPTW



c201 Fitness : 153988,96 - Nb Vehicles : 15



c201 Fitness : 30591,56 - Nb Vehicles : 3

<https://www.sintef.no/projectweb/top/vrptw/solomon-benchmark/documentation/>

PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

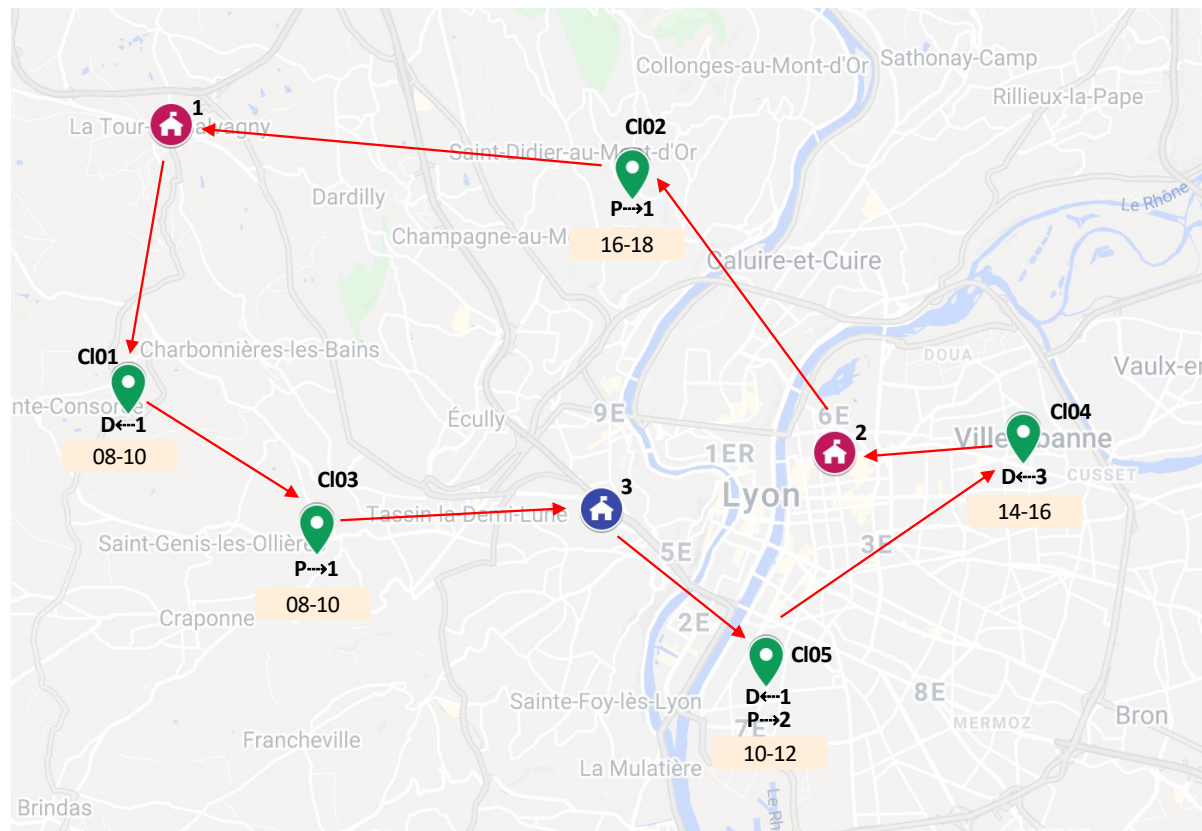
Vehicle Routing Problem with Pickup, Delivery and Time Windows

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```

(Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.C
  )

```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$


Créneaux

08-10

10-12

12-14

14-16

16-18

Etant donné une liste de clients, trouver la tournée optimale en termes de distance

P→2 Pickup vers 2

D←1 Delivery depuis 1

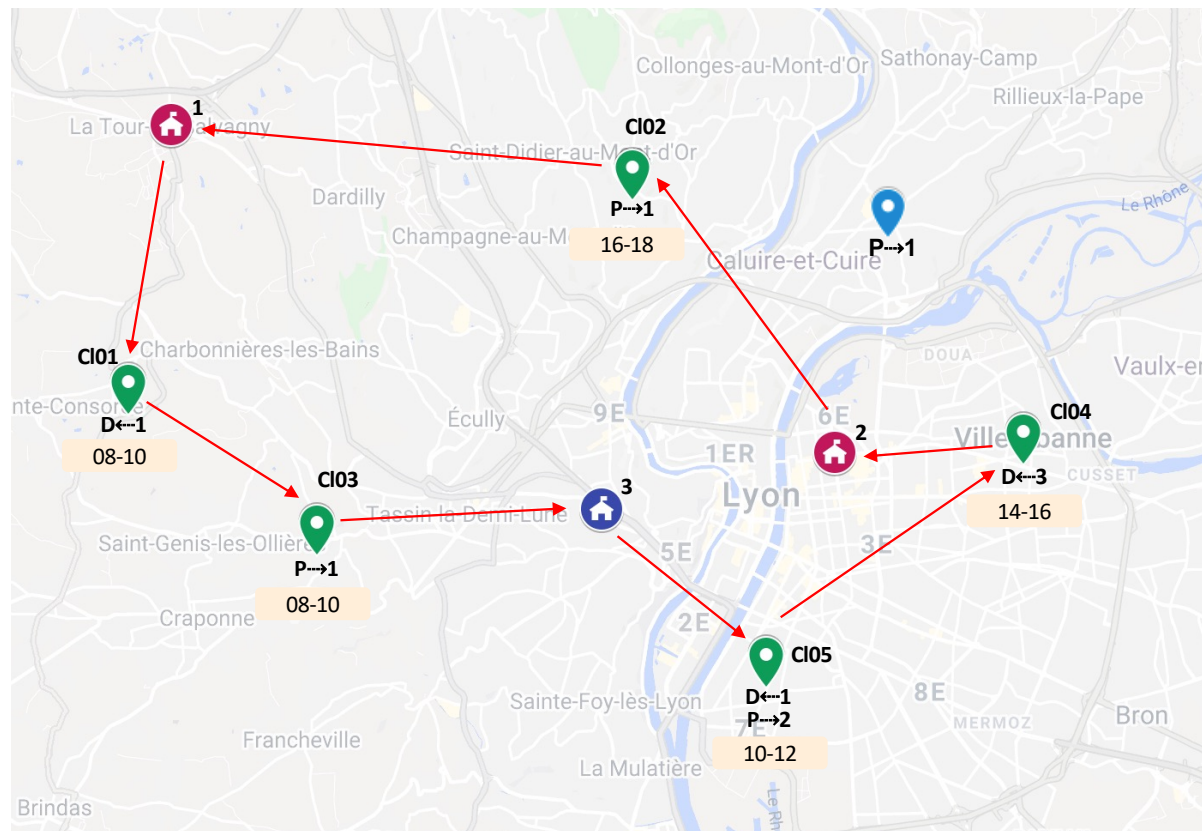
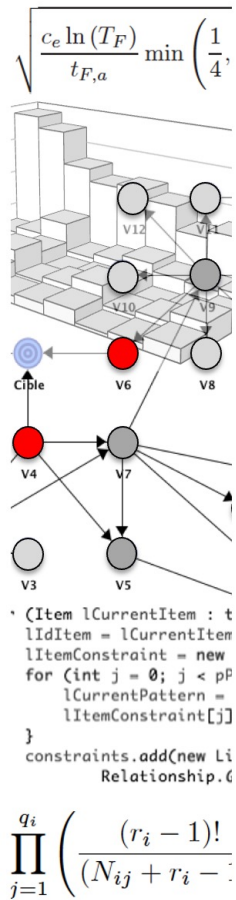
Pressing

Cordonnerie

il faut ajouter des contraintes liées à la capacité

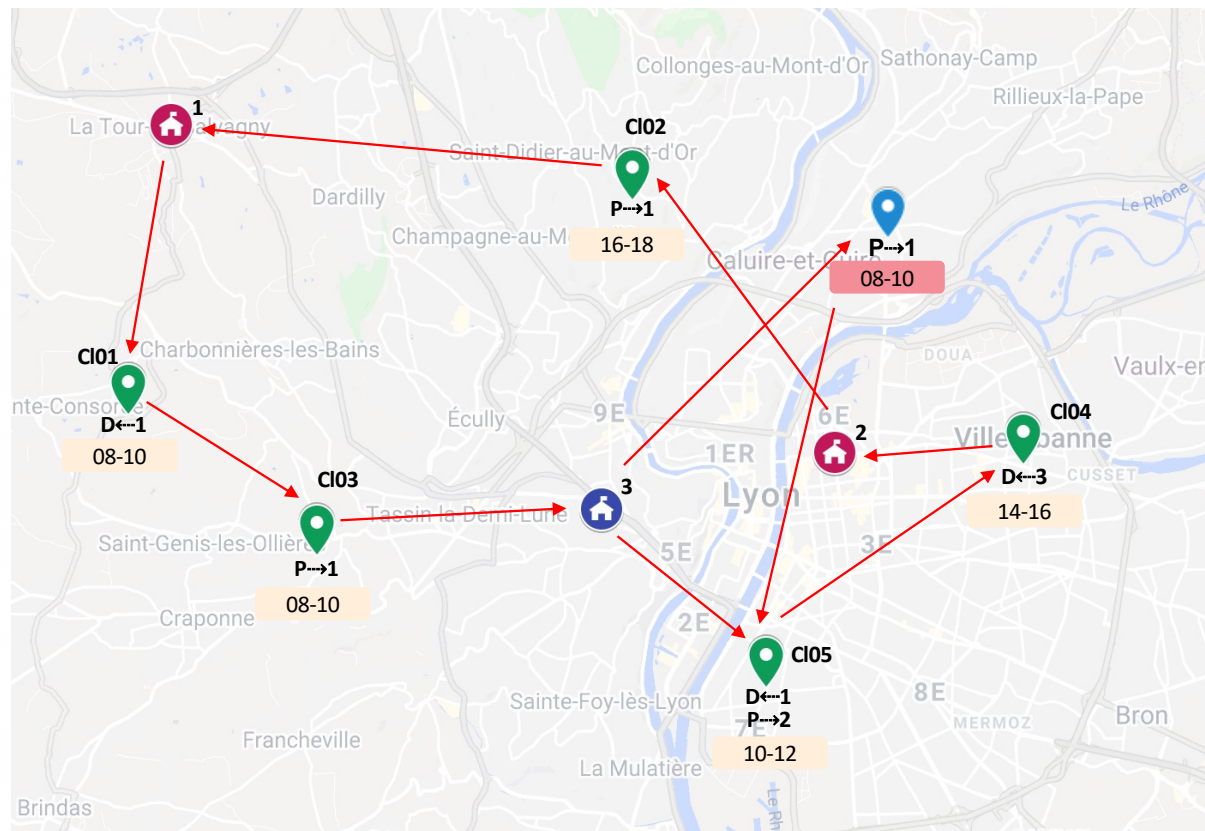
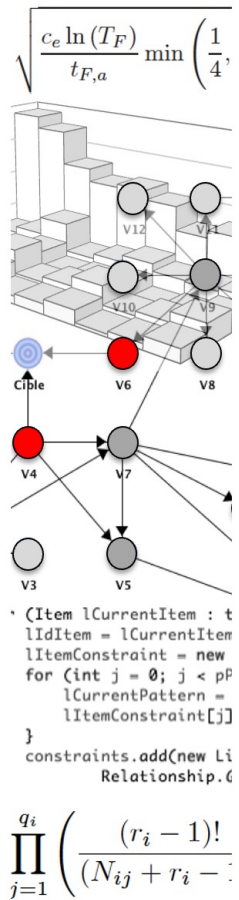
PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Positionner un nouveau client dans la tournée en testant chaque créneau



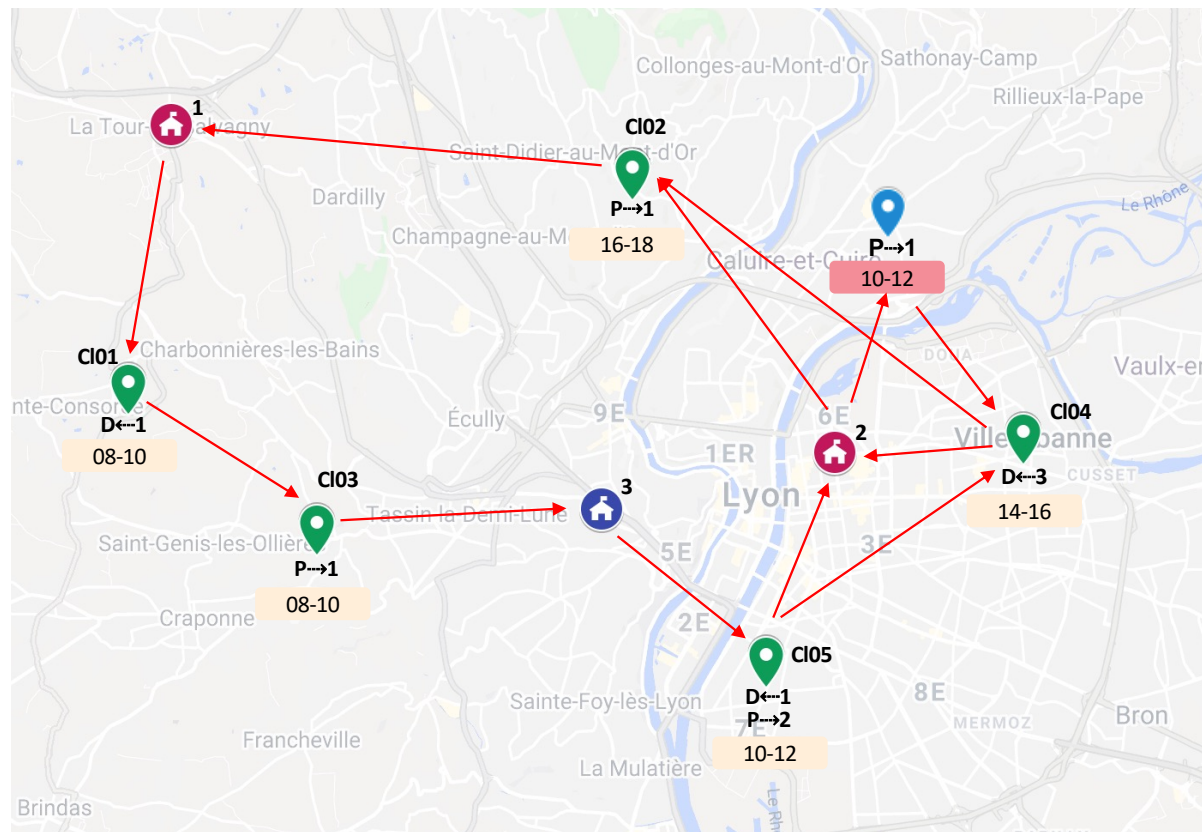
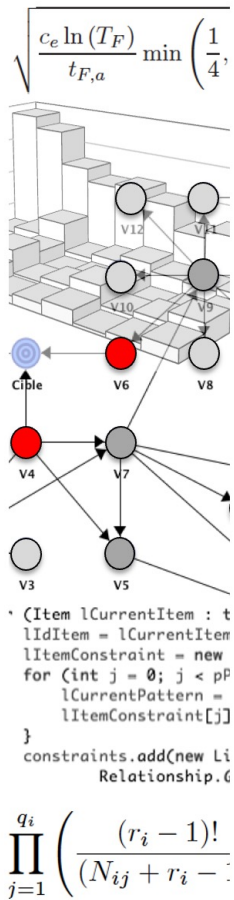
PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Positionner un nouveau client dans la tournée en testant chaque créneau



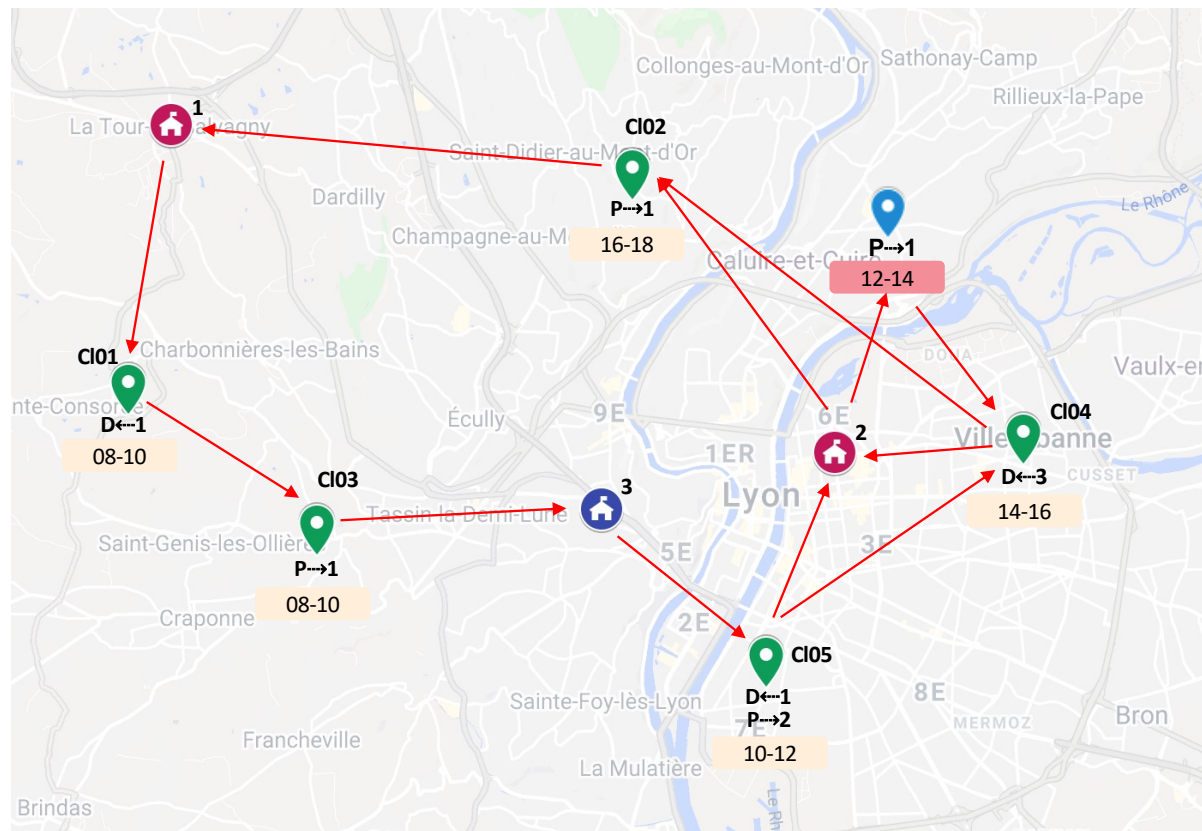
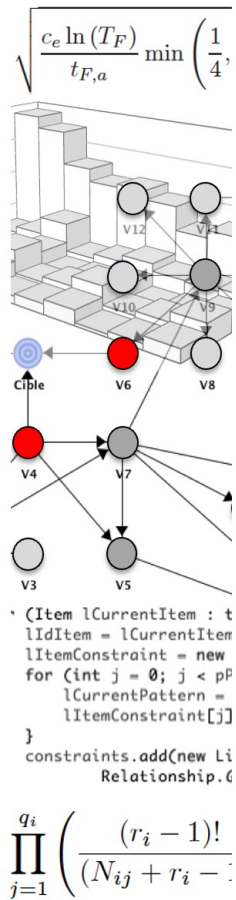
PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Positionner un nouveau client dans la tournée en testant chaque créneau



PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Positionner un nouveau client dans la tournée en testant chaque créneau

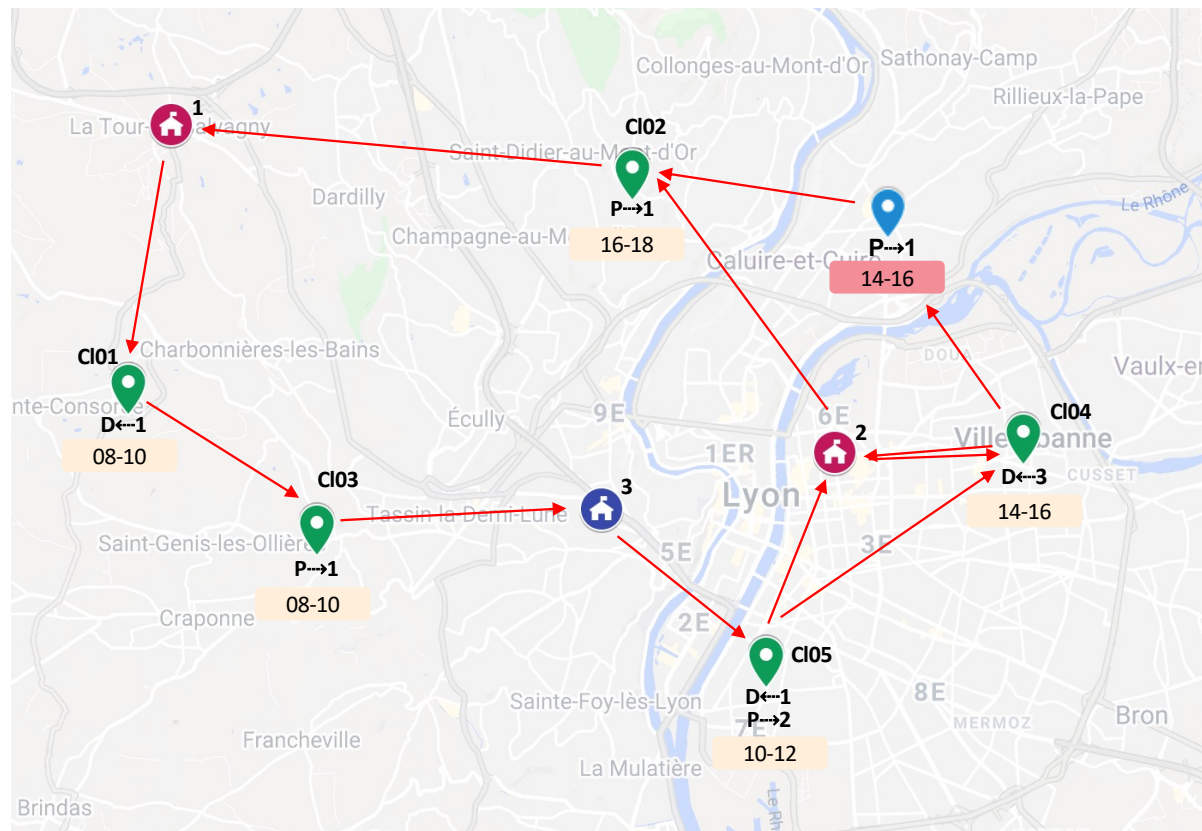
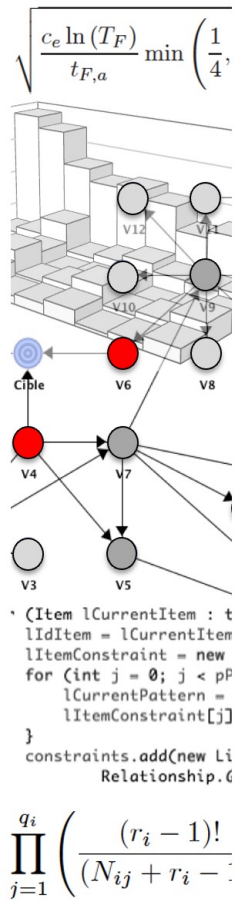


Créneaux	Δ Distance
08-10	27 %
10-12	13%
12-14	13%
14-16	
16-18	

- P→2 Pickup vers 2
- D←1 Delivery depuis 1
- Pressing
- Cordonnerie

PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Positionner un nouveau client dans la tournée en testant chaque créneau

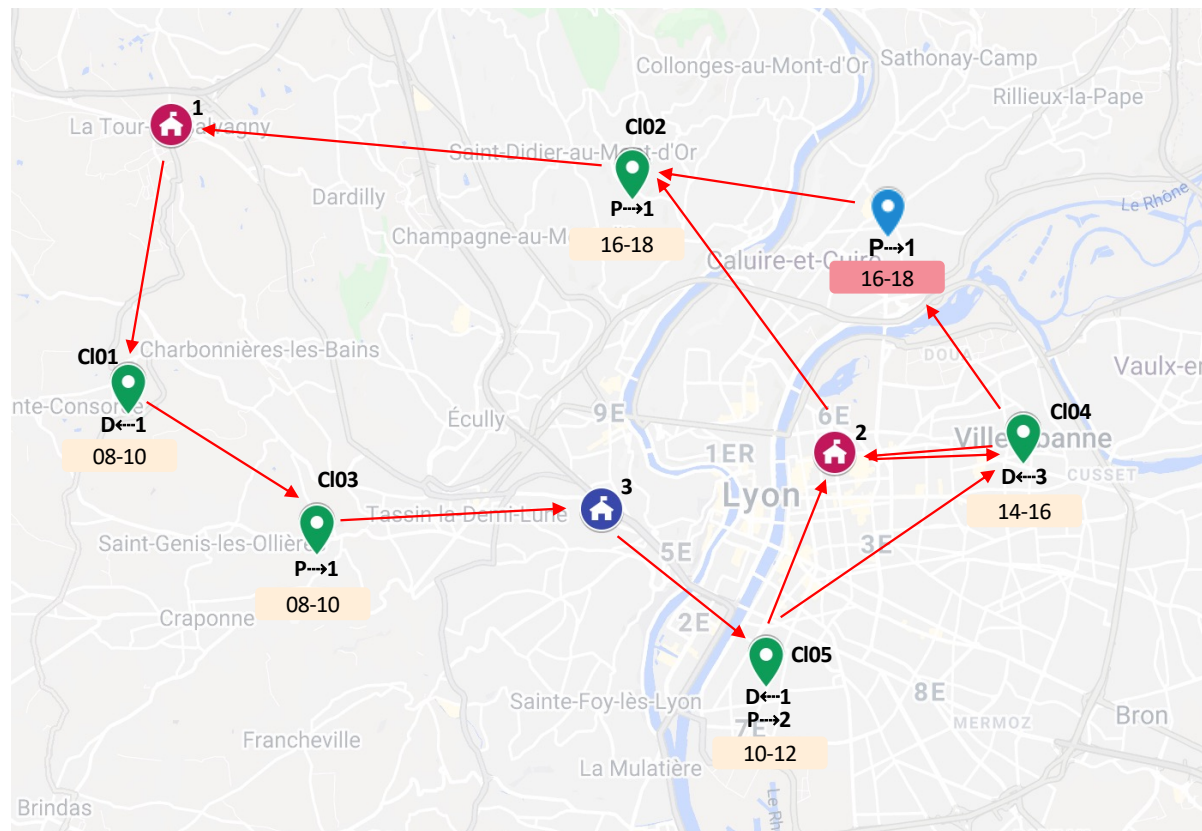
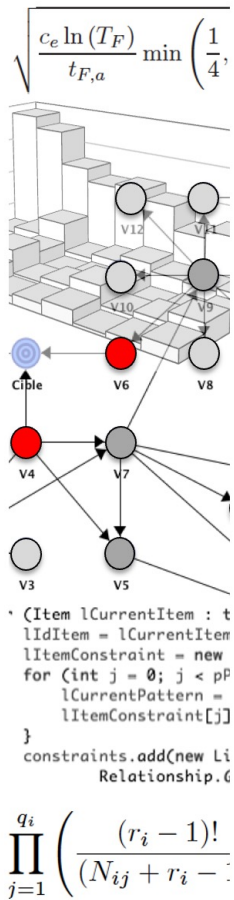


Créneaux	Δ Distance
08-10	27 %
10-12	13%
12-14	13%
14-16	3%
16-18	

- P->2 Pickup vers 2
- D<-1 Delivery depuis 1
- Pressing
- Cordonnerie

PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Positionner un nouveau client dans la tournée en testant chaque créneau



PROBLÈME DU VOYAGEUR DE COMMERCE – TRAVELING SALESMAN PROBLEM

Positionner un nouveau client dans la tournée en testant chaque créneau

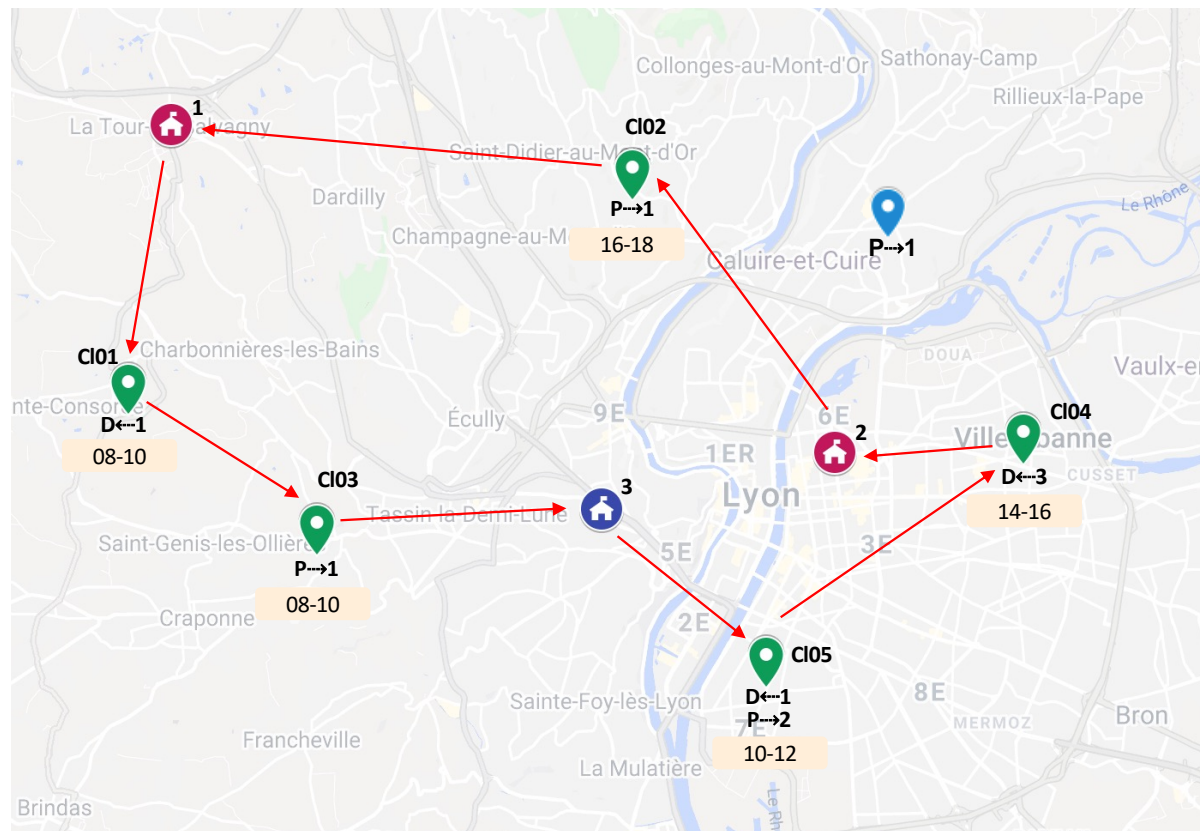
$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```

(Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.C
  )

```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$



Créneaux Δ Distance

08-10	27 %
10-12	13%
12-14	13%
14-16	3%
16-18	3%

Fixer un prix pour chaque créneau réalisable en fonction du Delta des distances

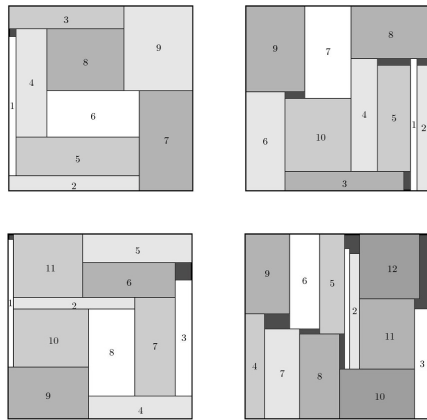
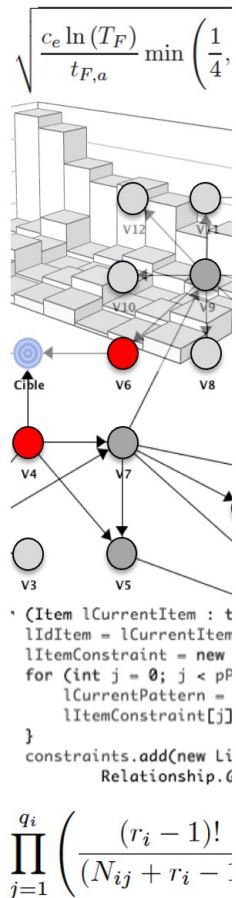
P→2 Pickup vers 2
D←1 Delivery depuis 1

Pressing

Cordonnerie

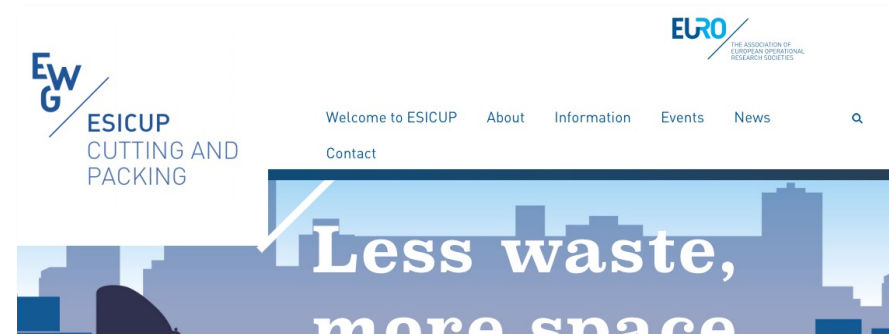
BIN PACKING PROBLEM

Soient un nombre infini de boîtes de taille C et n articles



Objectif : trouver un rangement de l'ensemble des articles qui minimise le nombre de boîtes (« bins ») utilisées

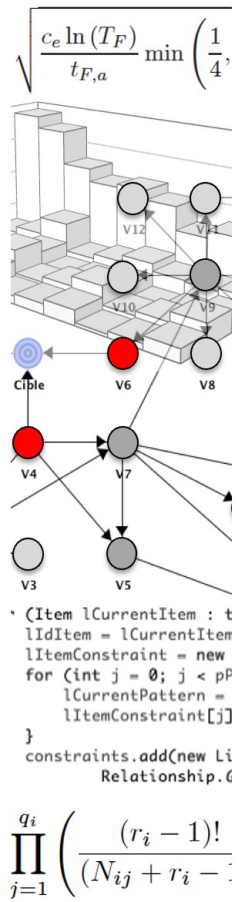
Contraintes : la somme des tailles des articles affectés à une boîte doit être inférieure ou égale à C



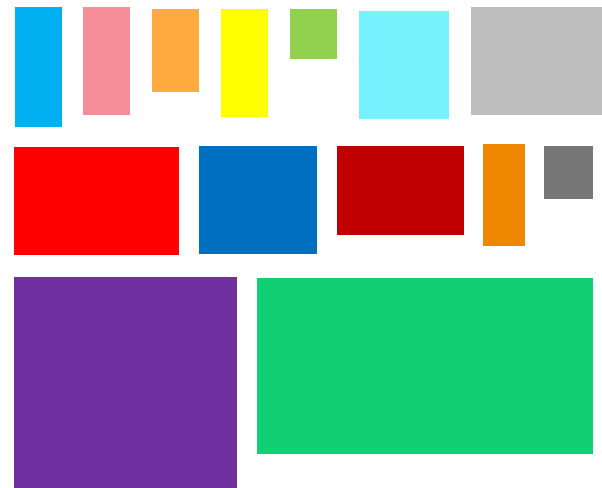
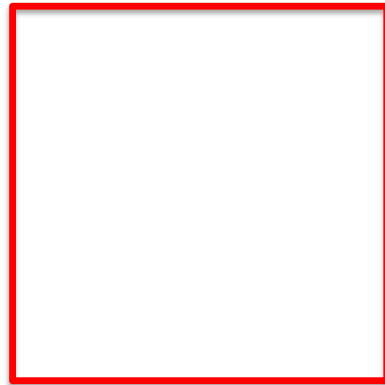
<https://www.euro-online.org/websites/esicup/>

BIN PACKING PROBLEM

Exemple « 2D rectangular bin packing »



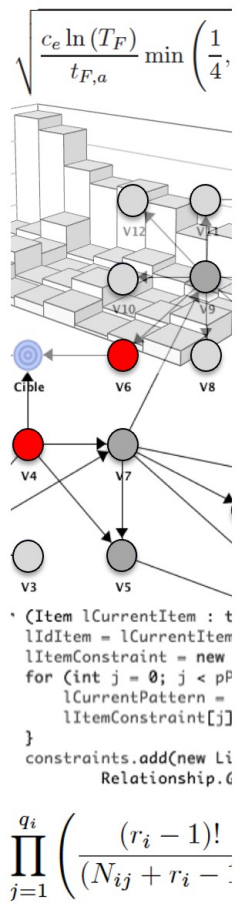
Bin



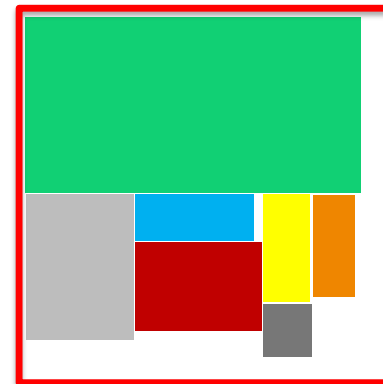
Minimiser le nombre de « bins » à utiliser pour mettre les 14 objets rectangulaires

BIN PACKING PROBLEM

Exemple « 2D rectangular bin packing »



Bin



Minimiser le nombre de « bins » à utiliser pour mettre les 10 objets rectangulaires

BIN PACKING PROBLEM

Exemple « 1D bin packing »

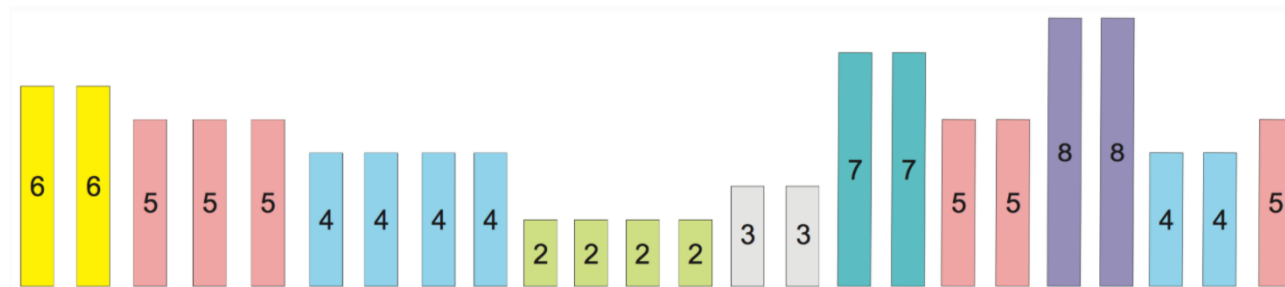
$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```

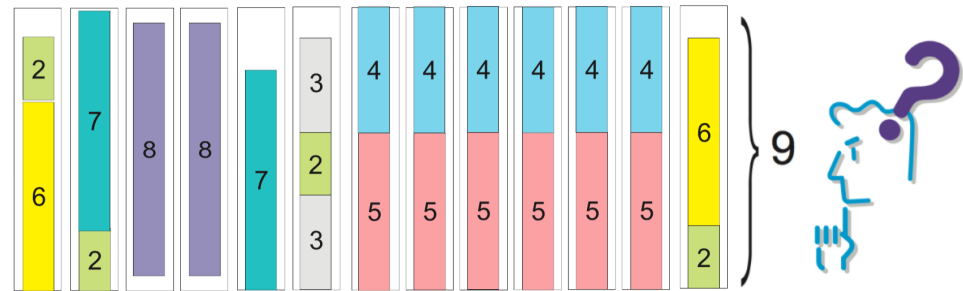
(Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.6

```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$



<https://scipbook.readthedocs.io/en/latest/bpp.html>

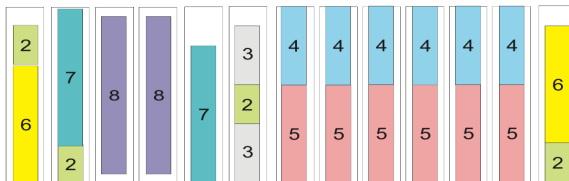


13 bins

BIN PACKING PROBLEM



1D bin packing



On dispose de n items ($I=\{1, \dots, n\}$) de taille monodimensionnelle (t_i : taille de l'item i) et d'un nombre illimité de boîtes, appelées « bins », de taille monodimensionnelle identique t .

L'objectif est de ranger tous les items dans des bins en utilisant le moins de bins possible. Il faut donc minimiser le nombre de bins utilisés.

1. Modéliser ce problème par un programme linéaire en nombres entiers (binaires).
2. Déterminer une borne inférieure du nombre de bins à utiliser.
3. Proposer des opérateurs de voisinage pour résoudre ce problème avec certaines métaheuristiques.

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```

        (Item lCurrentItem : t
        lIdItem = lCurrentItem
        lItemConstraint = new
        for (int j = 0; j < pP
        lCurrentPattern =
        lItemConstraint[j]
        }
        constraints.add(new Li
        Relationship.C
    
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$